NATIONAL ECONOMICS UNIVERSITY
**SCHOOL OF TECHNOLOGY**

# REPORT
# MACHINE LEARNING

**Topic:**

## FORECASTING DAILY PRODUCT VIEWS OF AN ONLINE CLOTHING STORE FROM CLICKSTREAM DATA

Group 3: Bui Quang Minh - 11247195
Ho Huyen Trang - 11247236
Ta Van Sang - 11247221
Class DS66B

Supervisor: Luong Van Thien

Hanoi, October 30, 2026

# REPORT EVALUATION FORM

(For the supervisor)

Evaluator: . . . . . .

Student name(s): . . . . . Student ID(s): . . . . . .

Report title: . . . . . .

. . . . . .

**Select the appropriate score for each criterion below:**

Very poor (1); Poor (2); Satisfactory (3); Good (4); Excellent (5)

| | | | | | | |
|---|---|---|---|---|---|---|
| **Combination of theory and practice (20)** | | | | | | |
| 1 | Clearly states the urgency and importance of the topic, the problems and hypotheses (including purpose and relevance) and the scope of application | 1 | 2 | 3 | 4 | 5 |
| 2 | Reviews the most recent research results (national/international) | 1 | 2 | 3 | 4 | 5 |
| 3 | Describes the research/problem-solving method clearly and in detail | 1 | 2 | 3 | 4 | 5 |
| 4 | Provides simulation/experimental results and presents them clearly | 1 | 2 | 3 | 4 | 5 |
| **Ability to analyse and evaluate results (15)** | | | | | | |
| 5 | Clear work plan, including objectives and methods, systematically based on the theoretical study | 1 | 2 | 3 | 4 | 5 |
| 6 | Results are presented logically and understandably; all results are adequately analysed and evaluated | 1 | 2 | 3 | 4 | 5 |
| 7 | In the conclusion, the authors point out the differences (if any) between the results and the initial objectives, and argue for feasible future directions | 1 | 2 | 3 | 4 | 5 |
| **Writing skills (10)** | | | | | | |
| 8 | The report follows the required template with a logical and well-presented chapter structure (clear, titled, numbered and referenced tables and figures; consistent margins; correct spacing after full stops and commas, etc.), has chapter introductions and summaries, lists references and cites them correctly | 1 | 2 | 3 | 4 | 5 |
| 9 | Excellent writing (correct sentence structure, scientific style, logical and well-founded arguments, appropriate vocabulary, etc.) | 1 | 2 | 3 | 4 | 5 |
| **Scientific research achievements (5)** *(choose 1 of 3 cases)* | | | | | | |
| 10a | Has a published or accepted scientific paper / won at least third prize in student research at school level / has a scientific award (international or national) of third prize or higher / has a registered patent or invention | 5 | | | | |
| 10b | Presented at a school-level student research conference without a third prize or higher / received a consolation prize in other national or international contests in the field (e.g. TI contest) | 2 | | | | |
| 10c | No scientific research achievements | 0 | | | | |
| **Total score** | | /50 | | | | |
| **Total score converted to a 10-point scale** | | | | | | |

***Other comments*** *(on the students' attitude and working spirit)*

.....

.....

.....

.....

.....

.....

Date: ... / ... / 20...

**Evaluator**

(Signature and full name)

# REPORT EVALUATION FORM

(For the reviewer)

Evaluator: . . . . . .

Student name(s): . . . . . Student ID(s): . . . . . .

Report title: . . . . . .

. . . . . .

**Select the appropriate score for each criterion below:**

Very poor (1); Poor (2); Satisfactory (3); Good (4); Excellent (5)

| | | | | | | |
|---|---|---|---|---|---|---|
| **Combination of theory and practice (20)** | | | | | | |
| 1 | Clearly states the urgency and importance of the topic, the problems and hypotheses (including purpose and relevance) and the scope of application | 1 | 2 | 3 | 4 | 5 |
| 2 | Reviews the most recent research results (national/international) | 1 | 2 | 3 | 4 | 5 |
| 3 | Describes the research/problem-solving method clearly and in detail | 1 | 2 | 3 | 4 | 5 |
| 4 | Provides simulation/experimental results and presents them clearly | 1 | 2 | 3 | 4 | 5 |
| **Ability to analyse and evaluate results (15)** | | | | | | |
| 5 | Clear work plan, including objectives and methods, systematically based on the theoretical study | 1 | 2 | 3 | 4 | 5 |
| 6 | Results are presented logically and understandably; all results are adequately analysed and evaluated | 1 | 2 | 3 | 4 | 5 |
| 7 | In the conclusion, the authors point out the differences (if any) between the results and the initial objectives, and argue for feasible future directions | 1 | 2 | 3 | 4 | 5 |
| **Writing skills (10)** | | | | | | |
| 8 | The report follows the required template with a logical and well-presented chapter structure (clear, titled, numbered and referenced tables and figures; consistent margins; correct spacing after full stops and commas, etc.), has chapter introductions and summaries, lists references and cites them correctly | 1 | 2 | 3 | 4 | 5 |
| 9 | Excellent writing (correct sentence structure, scientific style, logical and well-founded arguments, appropriate vocabulary, etc.) | 1 | 2 | 3 | 4 | 5 |
| **Scientific research achievements (5)** *(choose 1 of 3 cases)* | | | | | | |
| 10a | Has a published or accepted scientific paper / won at least third prize in student research at school level / has a scientific award (international or national) of third prize or higher / has a registered patent or invention | 5 | | | | |
| 10b | Presented at a school-level student research conference without a third prize or higher / received a consolation prize in other national or international contests in the field (e.g. TI contest) | 2 | | | | |
| 10c | No scientific research achievements | 0 | | | | |
| **Total score** | | /50 | | | | |
| **Total score converted to a 10-point scale** | | | | | | |

***Other comments of the reviewer***

.....

.....

.....

.....

.....

.....

Date: ... / ... / 20...

**Evaluator**

(Signature and full name)

# PREFACE

Online retailers record every click that a visitor makes on their websites. These clickstream logs are generated continuously and at almost no cost, yet in many small and medium-sized shops they are rarely turned into decisions. A simple question that a shop manager faces every evening –*which products will attract the most attention tomorrow?* can be answered from exactly this kind of data. A reliable answer helps the shop decide which products to feature on the home page, how to allocate advertising and which products are gaining or losing interest.

This report applies the machine learning methods studied in our course to that question. We use the public *e-shop clothing 2008* clickstream dataset, which contains 165,474 product views recorded by an online clothing store between April and August 2008. Before building any model, we re-examine a task that is frequently attempted on this dataset in public notebooks – classifying the price-level variable `price2` and show experimentally why its high scores do not reflect real predictive value. We then reformulate the problem as forecasting the number of views of each product on the next day, construct a product–day table, design features that use only past information, and compare four simple baselines with Linear Regression, Ridge Regression, K-Nearest Neighbours, Decision Tree and Random Forest under a strict time-based evaluation. Finally, we test an alternative classification formulation that allows Logistic Regression and Naive Bayes to be applied in the right place.

Throughout the work we tried to answer not only "which model scores best?" but also "can this score be trusted?". For this reason the report pays particular attention to data leakage, to the choice of a strong baseline and to the honest reporting of limitations.

We would like to express our sincere gratitude to **Luong Van Thien** for the guidance and valuable feedback throughout the course and this project. We also thank the School of Technology, National Economics University, for providing the learning environment. Because of limited time and knowledge, the report may still contain shortcomings; we would be grateful to receive comments from lecturers and readers to improve it further.

# DECLARATION

We, the members of Group 3, class DS66B – Bui Quang Minh, Ho Huyen Trang and Ta Van Sang – under the supervision of **Luong Van Thien**, have carried out the project *"Forecasting daily product views of an online clothing store from clickstream data"*.

We declare that the content of this report is the result of our own study and work. All results reported in Chapters 3 and 4 are produced by the accompanying notebook `DS66B_group_3.ipynb` on the original data file, and every figure and table can be reproduced by running that notebook from top to bottom. All cited documents and data sources are listed in the References section.

We take full responsibility for the honesty and content of this report.

Hanoi, October 30, 2026

**On behalf of the group**

**Bui Quang Minh**

# TEAM CONTRIBUTIONS

Table 0.1 summarises the contribution of each member. The task names follow the parts of the notebook `DS66B_group_3.ipynb` and the chapters of this report.

| Member | Main tasks | Contribution |
|---|---|---|
| Ho Huyen Trang | Data loading and quality checks (Parts 1–2), re-examination of the `price2` approach (Part 3); Chapters 1 and 2 of the report | 33.33% |
| Ta Van Sang | Product–day table, exploratory analysis and feature engineering (Parts 4–5), leakage checks; Chapter 3 of the report | 33.33% |
| Bui Quang Minh | Models, hyperparameter selection and evaluation (Parts 6–9), classification view; Chapter 4, conclusion and slides | 33.33% |

**Table 0.1 Contribution of each member.**

# TABLE OF CONTENTS

**LIST OF ABBREVIATIONS AND SYMBOLS** i

**LIST OF FIGURES** iii

**LIST OF TABLES** iv

**ABSTRACT** v

Problem statement . . . . . 1

Objectives and contributions . . . . . 1

Structure of the report . . . . . 2

**CHAPTER 1. OVERVIEW OF THE PROBLEM** 4

1.1 Clickstream data and demand signals in e-commerce . . . . . 4

1.2 The e-shop clothing 2008 dataset . . . . . 4

1.3 Re-examining the common approach: classifying `price2` . . . . . 5

1.4 Challenges of forecasting daily product views . . . . . 6

1.4.1 Temporal order and data leakage . . . . . 6

1.4.2 A small count target with many zeros . . . . . 6

1.4.3 Short history and a short evaluation window . . . . . 6

1.4.4 Confounding by display position . . . . . 6

1.4.5 Data-quality issues . . . . . 7

1.4.6 Missing context . . . . . 7

1.5 Objectives and methodology . . . . . 7

1.5.1 Objectives of the project . . . . . 7

1.5.2 Methodology . . . . . 7

1.6 Chapter summary . . . . . 8

**CHAPTER 2. THEORETICAL BACKGROUND AND RELATED WORK** 9

2.1 Problem formulation . . . . . 9
2.2 Related work . . . . . 10
2.2.1 Demand forecasting in retail and e-commerce . . . . . 10
2.2.2 Naive and classical time-series methods . . . . . 10
2.2.3 Machine learning with lag features . . . . . 11
2.2.4 Classification-based formulations . . . . . 11
2.2.5 Deep learning . . . . . 11
2.2.6 Evaluation pitfalls . . . . . 11
2.3 Theoretical foundations . . . . . 12
2.3.1 Linear Regression . . . . . 12
2.3.2 Ridge Regression . . . . . 13
2.3.3 Standardisation . . . . . 13
2.3.4 One-hot encoding . . . . . 13
2.3.5 K-Nearest Neighbours . . . . . 14
2.3.6 Decision Tree . . . . . 14
2.3.7 Random Forest . . . . . 14
2.3.8 Logistic Regression . . . . . 15
2.3.9 Gaussian Naive Bayes . . . . . 15
2.4 Evaluation metrics . . . . . 15
2.4.1 Regression metrics . . . . . 15
2.4.2 Classification metrics . . . . . 16
2.5 Chapter summary . . . . . 16

**CHAPTER 3. METHODOLOGY** **17**

3.1 Data loading and first checks . . . . . 17
3.2 Data quality checks and cleaning . . . . . 18
3.2.1 Date column and calendar continuity . . . . . 18
3.2.2 Truncated last day . . . . . 18
3.2.3 Consistency of product attributes and correction of A18 . . . . 18
3.3 Design of the `price2` re-examination . . . . . 19

3.4 Building the product–day panel . . . . . 19
3.4.1 Complete grid with zero counts . . . . . 19
3.4.2 Product life cycle . . . . . 20
3.5 Exploratory analysis of the target . . . . . 20
3.5.1 Distribution. . . . . 20
3.5.2 Weekly cycle. . . . . 20
3.5.3 Category and page. . . . . 21
3.6 Feature engineering and leakage control . . . . . 22
3.6.1 Features . . . . . 22
3.6.2 Manual leakage check . . . . . 23
3.6.3 Warm-up period . . . . . 23
3.7 Encoding and time-based split . . . . . 24
3.8 Models and hyperparameter selection . . . . . 24
3.9 Final evaluation protocol . . . . . 25
3.10 Analysis protocol . . . . . 26
3.11 Classification reformulation . . . . . 27
3.12 Chapter summary . . . . . 27

**CHAPTER 4. EXPERIMENTS AND EVALUATION 28**

4.1 Experimental objectives . . . . . 28
4.2 Experimental setup . . . . . 28
4.2.1 Computing environment . . . . . 28
4.2.2 Dataset . . . . . 28
4.3 Experiment 0: re-examining the `price2` task . . . . . 29
4.4 Baselines and linear models . . . . . 30
4.4.1 Baselines . . . . . 30
4.4.2 Linear Regression . . . . . 30
4.4.3 Ridge Regression . . . . . 30
4.5 K-Nearest Neighbours . . . . . 31
4.6 Decision Tree and Random Forest . . . . . 32

4.7 Model comparison on the validation set . . . . . 33
4.8 Final results on the test set . . . . . 33
4.9 Reliability of the improvement . . . . . 34
4.10 Error analysis . . . . . 35
4.11 Feature importance . . . . . 36
4.12 Leakage stress test . . . . . 37
4.13 Classification view . . . . . 37
4.14 Discussion . . . . . 39
4.15 Chapter summary . . . . . 40

**CONCLUSION** **41**

**General conclusions** **41**

**REFERENCES** **43**

**APPENDIX. REPORT–NOTEBOOK CORRESPONDENCE** **46**

# LIST OF ABBREVIATIONS AND SYMBOLS

| | Full name | Meaning in this report |
|---|---|---|
| AI | Artificial Intelligence | — |
| ML | Machine Learning | Learning patterns from data |
| EDA | Exploratory Data Analysis | Visual and statistical study of the data |
| CSV | Comma-Separated Values | Format of the raw data file |
| UCI | University of California, Irvine | UCI Machine Learning Repository |
| **– Models** | | |
| LR | Linear Regression | Ordinary least-squares regression |
| OLS | Ordinary Least Squares | Fitting criterion of LR |
| KNN | K-Nearest Neighbours | Regressor/classifier based on similar past cases |
| DT | Decision Tree | CART regression/classification tree |
| CART | Classification and Regression Trees | Tree-growing algorithm used by scikit-learn |
| RF | Random Forest | Average of many bootstrapped trees |
| NB | Naive Bayes | Gaussian Naive Bayes classifier |
| **– Evaluation** | | |
| MAE | Mean Absolute Error | Main regression metric |
| RMSE | Root Mean Squared Error | Penalises large errors more |
| MAPE | Mean Absolute Percentage Error | Computed only where $y > 0$ |
| $R^2$ | Coefficient of determination | Share of variance explained |
| TP / FP | True / False Positive | Correct / wrong “high interest” alarms |
| TN / FN | True / False Negative | Correct rejections / missed cases |
| F1 | F1-score | Harmonic mean of Precision and Recall |
| **– Variables and symbols** | | |
| $p$, $d$ | Product, day | Indices of a product–day pair |

*Continued on next page*

*(Continued)*

| | **Full name** | **Meaning in this report** |
|---|---|---|
| $y_{p,d}$ | Target | Number of views of product $p$ on day $d$ (`views`) |
| $\hat{y}_{p,d}$ | Prediction | Forecast of $y_{p,d}$ |
| $\mathbf{x}_{p,d}$ | Feature vector | Inputs available before day $d$ |
| `lag_1`, `lag_7` | Lag features | Views 1 and 7 days earlier |
| `avg_7_days`, `avg_14_days` | Rolling means | Mean views over the previous 7 / 14 days |
| $n$ | Number of observations | — |
| $\alpha$ | Ridge penalty | Regularisation strength |
| $k$ | Number of neighbours | KNN hyperparameter |
| $\sigma(\cdot)$ | Sigmoid function | Logistic Regression link |
| $\mathbb{1}[\cdot]$ | Indicator function | 1 if the condition holds, otherwise 0 |

# LIST OF FIGURES

Figure 1.1 Overall pipeline of the project. . . . . 8

Figure 3.1 Total number of clicks per day. . . . . 18

Figure 3.2 Distribution of the target `views` (product–day pairs up to 15 July). 20

Figure 3.3 Average views of one product by day of the week (training period). 21

Figure 3.4 Average views per product per day by category. . . . . 21

Figure 3.5 Average views per product per day by page of the website. . . . . 22

Figure 4.1 Training and validation MAE of the Decision Tree by depth. . . 32

Figure 4.2 Actual versus predicted views on the test set. . . . . 35

Figure 4.3 Total daily views on the test period. . . . . 36

Figure 4.4 Ten most important features of the Random Forest (impurity-based importance). . . . . 37

# LIST OF TABLES


Table 0.1 Contribution of each member. . . . . 8

Table 1.1 Variables of the e-shop clothing 2008 dataset (after renaming). . . 5

Table 2.1 Inputs and outputs of the forecasting problem. . . . . 10

Table 2.2 Families of demand-forecasting methods. . . . . 12

Table 3.1 Correspondence between notebook parts and report sections. . . . 17

Table 3.2 Clicks by first letter of product code and category. . . . . 19

Table 3.3 Number of products of each category on each page. . . . . 22

Table 3.4 Features used to forecast the views of product $p$ on day $d$. . . . . 23

Table 3.5 Time-based split of the modelling table. . . . . 24

Table 3.6 Models, inputs and hyperparameter grids. . . . . 25

Table 4.1 Size of the data at each stage. . . . . 29

Table 4.2 Accuracy of decision trees predicting price2. . . . . 29

Table 4.3 Validation MAE of Ridge Regression for different values of $\alpha$. . . 30

Table 4.4 Correlation between the history features (training set). . . . . 31

Table 4.5 Coefficients of the history features. . . . . 31

Table 4.6 Validation MAE of the KNN regressor. . . . . 31

Table 4.7 Training and validation MAE of the Decision Tree for each depth. 32

Table 4.8 Validation MAE of the Random Forest for each depth. . . . . 33

Table 4.9 Validation results of all baselines and models. . . . . 33

Table 4.10 Test results of all baselines and models. . . . . 34

Table 4.11 Daily MAE of the Random Forest and of the strongest baseline. . 34

Table 4.12 Test error of the Random Forest by category. . . . . 36

Table 4.13 Effect of forgetting the one-day shift on the test scores. . . . . 37

Table 4.14 Validation results of the classification formulation. . . . . 38

Table 4.15 Test results of the classification formulation. . . . . 38

Table 5.1 Model and feature-set labels in the notebook and in the report. . . 46

# ABSTRACT

Clickstream logs of online shops record every product view, but they are seldom used to support daily merchandising decisions. This report studies the public *e-shop clothing 2008* dataset (165,474 clicks on 217 products, 1 April–13 August 2008) and asks how many times each product will be viewed on the next day.

We first re-examine a task that is frequently attempted on this dataset – classifying the price-level indicator `price2`. Three experiments show that its high scores are not meaningful: keeping `price` as an input gives 100% accuracy because the label is a deterministic function of the input (target leakage); removing `price` still gives 92.91% under a random split because the same 217 products appear in both training and test data; and on unseen products accuracy falls to 44.1%, below the 51.2% obtained by always predicting the majority class.

We therefore reformulate the problem as one-step-ahead forecasting of daily product views. After correcting two data-quality issues (a truncated last day and one inconsistent category label), we build a 217-product $\times$ 134-day panel, create lag, rolling-mean and day-of-week features that use only past information, and evaluate models with a strictly time-ordered train/validation/test split. Four naive baselines are compared with Linear Regression, Ridge Regression, K-Nearest Neighbours, Decision Tree and Random Forest, with all hyperparameters chosen on the validation period. Random Forest obtains the lowest validation error and reaches a test MAE of 2.023 views per product per day ($R^2 = 0.711$), 4.8% lower than the strongest baseline (the 14-day average, MAE 2.126). However, it beats this baseline on only 7 of the 12 test days, and a coin-flip simulation shows that such a result would occur by chance about 39% of the time; the improvement is therefore promising but not yet conclusive. Most of the accuracy comes from well-designed history features rather than from model complexity. An alternative classification formulation ("will a product receive at least 7 views tomorrow?") confirms this: classifiers, including Logistic Regression and Naive Bayes, improve F1 by only about 0.01 over a simple rule, and the regression model with a threshold performs equally well.

# INTRODUCTION

## Problem statement

E-commerce has made it possible for a shop to observe the interest of its visitors directly: every time a visitor opens a product page, the web server writes a line into a clickstream log. Aggregated over a day, these lines tell the shop how much attention each product receives. Anticipating this attention one day in advance is useful for routine decisions such as choosing the products to feature on the home page, allocating advertising budgets and noticing products whose popularity is rising or falling. In retail practice, such forecasts are usually produced by simple rules ("tomorrow will look like the recent average"), and a natural question is whether machine learning can do better.

The *e-shop clothing 2008* dataset [1, 2] is a public clickstream log of an online clothing store. It is widely used in teaching and in public notebooks, where the most common task is to classify the variable `price2`, which indicates whether a product is more expensive than the average of its category. As shown in this report, this task suffers from target leakage and from memorisation of a small set of products, and its answer is already known to the shop. A more meaningful question is to forecast the number of views of each product on the next day. This question, however, raises its own difficulties: the observations are ordered in time, so any use of future information inflates the scores; the target is a small count with many zeros; the history is short; and the attention a product receives is strongly influenced by where it is displayed on the website.

## Objectives and contributions

The objective of this project is to apply the machine learning methods studied in the course – Linear Regression, Ridge Regression, K-Nearest Neighbours, Decision Tree, Random Forest, Logistic Regression and -Naive Bayes – to a concrete business problem in a methodologically correct way, and to judge honestly how much they add over simple rules. Every result in this report is produced by the accompanying notebook `DS66B_group_3.ipynb`.

The main contributions are:

- A critical re-examination of the common `price2` classification task, with three experiments that separate target leakage, memorisation of products and genuine

generalisation to unseen products.

- A reformulation of the problem as one-step-ahead forecasting of daily product views, including the construction of a complete product–day panel and two data-quality corrections (a truncated last day and an inconsistent category label).

- A leakage-safe feature design (lags, rolling means computed after a one-day shift within each product, day of week), verified by a manual calculation and by a deliberate "wrong-shift" experiment.

- A controlled comparison of four baselines and five regression models with a time-ordered train/validation/test split, hyperparameters chosen on validation and a single final evaluation on the test period, together with a day-by-day reliability check and an error analysis.

- An alternative classification formulation that applies Logistic Regression and Naive Bayes where they belong and shows that it brings no gain over the regression formulation.

## Structure of the report

The report consists of four chapters followed by a conclusion:

***Chapter 1: Overview of the problem.*** This chapter introduces clickstream data and its use for demand signals, describes the dataset, explains why the common `price2` task is not meaningful, lists the specific challenges of forecasting daily product views and states the objectives and methodology of the project.

***Chapter 2: Theoretical background and related work.*** This chapter formulates the problem mathematically, reviews the main families of demand-forecasting methods (naive and statistical time-series methods, machine learning with lag features, classification-based formulations and deep learning) and the main evaluation pitfalls, and presents the theory of every algorithm and metric used in the project.

***Chapter 3: Methodology.*** This chapter describes the full pipeline implemented in the notebook: data cleaning, the product–day panel, exploratory analysis of the target, feature engineering and leakage checks, encoding and time-based splitting, the models and their hyperparameter grids, the evaluation protocol and the classification reformulation.

***Chapter 4: Experiments and evaluation.*** This chapter reports all experimen-

tal results: the re-examination of `price2`, the tuning of every model, the validation and test comparisons, a day-by-day reliability check, the error analysis, feature importance, the leakage stress test and the classification view, followed by a discussion of what drives accuracy and of the limitations of the study.

# CHAPTER 1. OVERVIEW OF THE PROBLEM

This chapter introduces the problem studied in the report. We first explain what clickstream data are and why the daily number of product views is a useful demand signal for an online shop. We then describe the *e-shop clothing 2008* dataset, discuss why the task most often attempted on it – classifying `price2` – has little practical value, and list the specific challenges of forecasting daily product views. The chapter ends with the objectives and the overall methodology of the project.

## 1.1 Clickstream data and demand signals in e-commerce

A *clickstream* is the sequence of pages that a visitor opens during a visit (a *session*) to a website. Online shops record clickstreams automatically: each time a product page is opened, one line is written into the log with the time, the session identifier and the product shown. Unlike sales records, which exist only when a purchase is completed, clickstream records exist for every product that attracts attention. Counted per product and per day, they form a continuous signal of interest in each item.

Forecasting demand is a central task in retail [3]. Online retailers in particular use forecasts of product-level demand to support pricing, assortment and merchandising decisions [4]. In a small online shop, the most immediate use of a one-day-ahead forecast of product views is operational: choosing which products to feature on the home page or in a category page, deciding where to spend the next day's advertising budget, and spotting products whose popularity changes. These decisions are taken every day and are usually based on simple rules such as "tomorrow will be similar to the recent average". The question addressed in this report is whether machine learning models can improve on such rules when they are evaluated correctly.

It is important to state from the start what such forecasts *cannot* do. Product views are a proxy for demand, not sales: a view does not mean a purchase. A one-day horizon is also too short for purchasing stock, which requires forecasts several days or weeks ahead. The application considered here is therefore daily operational merchandising, not inventory planning.

## 1.2 The e-shop clothing 2008 dataset

The dataset was collected from an online store offering clothing for pregnant women and was analysed by Lapczynski and Bialowas [1]; it is distributed through the UCI Machine Learning Repository [2]. The file used in this project contains **165,474 rows and 14 columns**, separated by semicolons. The observations cover **135 consecutive days, from 1 April to 13 August 2008**, with **217 products** and **24,026 sessions**.

The data contain no missing values and no duplicated rows.

Inspecting all rows of a single session shows that the column `order` runs from 1 to the number of rows of that session (for session 1, from 1 to 9). Hence **each row represents one click on one product page**. The dataset contains **no information about purchases**; the quantity that can be measured is therefore the number of views. Table 1.1 lists the variables after the columns are renamed in the notebook.

| Name in notebook | Description | Values |
|---|---|---|
| `year`, `month`, `day` | Date of the click | 2008; months 4–8 |
| `order` | Sequence number of the click within the session | 1–195 |
| `country` | Country of origin of the IP address | codes 1–47 (29 = Poland) |
| `session` | Session identifier | 24,026 sessions |
| `category` | Main product category | 1 trousers, 2 skirts, 3 blouses, 4 sale |
| `product` | Product code | 217 codes (e.g. A13) |
| `colour` | Colour of the product | codes 1–14 |
| `location` | Position of the photo on the page | 1–6 (top left … bottom right) |
| `photo` | Model photography | 1 en face, 2 profile |
| `price` | Price in US dollars | 18–82 |
| `price2` | Whether the price is higher than the average price of the category | 1 yes, 2 no |
| `page` | Page number of the product within the website | 1–5 |

**Table 1.1 Variables of the e-shop clothing 2008 dataset (after renaming).**

Most of these variables are *codes* rather than quantities. For example, colour 12 (red) is not "larger" than colour 3 (blue). This observation determines how the variables are encoded in Chapter 3.

## 1.3 Re-examining the common approach: classifying `price2`

Public notebooks that use this dataset frequently treat it as a classification problem in which each click is labelled by `price2` and the product attributes are used as inputs. Such notebooks usually report very high accuracy. Before adopting this framing, we checked how the product attributes behave over time. In the whole dataset, **every product keeps exactly one price, one colour, one photo position, one photo type, one page and one value of** `price2` during the five months. Consequently:

- `price2` is a deterministic function of `price` and `category`. If `price` is an input, the label is contained in the input, which is a textbook case of target leakage [5].

- Without `price`, the remaining attributes still identify most of the 217 products. If clicks are split at random, the same product appears in both the training and the test data, so a model can simply memorise products.

- Even a perfect classifier would be useless to the shop, which already knows the prices of its own products.

Section 4.3 quantifies these three points: accuracy is 100% with `price`, 92.91% without `price` under a random split, but only 44.1% when the model is tested on products it has never seen, below the 51.2% obtained by always predicting the majority class. The quantity that *changes every day* and that the shop does *not* know in advance is the number of views of each product. This is the problem studied in the rest of the report.

## 1.4 Challenges of forecasting daily product views

### *1.4.1 Temporal order and data leakage*

The forecast for day $d$ is made at the end of day $d-1$. Any input that uses information from day $d$ or later is unavailable in practice. Such leakage can be introduced in subtle ways, for example by computing a moving average that includes the current day, by splitting the data at random instead of by time, or by fitting a scaler on the whole dataset. Leakage produces optimistic scores that disappear in deployment [5], and standard random cross-validation is not appropriate for time-ordered data [6].

### *1.4.2 A small count target with many zeros*

The number of views of a product on a day is a non-negative integer. In the training period the median is 4 views, the mean 5.77 views and the maximum 79 views, and about 10% of the product–day pairs have zero views (Section 3.5). Percentage errors are undefined when the actual value is zero and are inflated when it is small [7], so the choice of metric matters.

### *1.4.3 Short history and a short evaluation window*

The log covers only about four and a half months. After setting aside 16 days for validation and 12 days for testing, the final evaluation is based on a short period, which limits how strongly conclusions can be drawn.

### *1.4.4 Confounding by display position*

Products on page 1 receive several times more views than products on page 5, and the categories are not spread evenly over pages. Differences between categories may

therefore reflect *where* products are shown rather than *what* they are, and the observational data cannot answer causal questions such as "how many views would a product get if it were moved to page 1?".

#### *1.4.5 Data-quality issues*

Real logs contain errors. In this dataset the last day is truncated and one product is recorded under two different categories (Section 3.2). Such errors must be detected and corrected with evidence, not by assumption.

#### *1.4.6 Missing context*

The data contain no information on promotions, holidays, advertising campaigns or weather. Sudden shifts in traffic caused by such events cannot be anticipated from the history of views alone.

### 1.5 Objectives and methodology

#### *1.5.1 Objectives of the project*

The project pursues four objectives:

- Show, with experiments, why the common `price2` task is not a meaningful predictive problem.
- Formulate the forecasting of daily product views as a supervised regression problem that uses only information available before the forecast day.
- Compare the regression algorithms studied in the course with strong naive baselines under a correct time-based protocol, and quantify how much they improve and how reliable the improvement is.
- Check whether a classification formulation, which allows Logistic Regression and Naive Bayes to be used, offers any advantage over the regression formulation.

#### *1.5.2 Methodology*

The methodology, implemented step by step in the notebook, is summarised in Figure 1.1. All decisions on features, models and hyperparameters are taken on the training and validation periods; the test period is used only once, at the end.

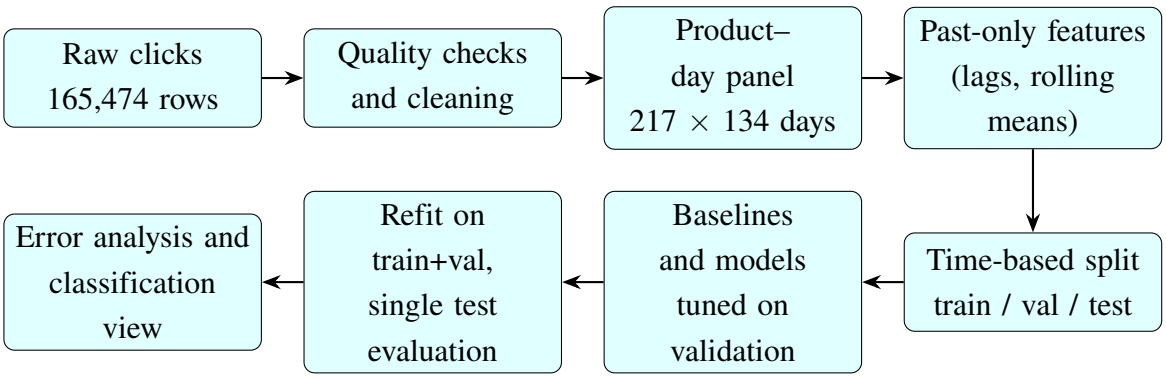



**Figure 1.1 Overall pipeline of the project.**

## 1.6 Chapter summary

This chapter has introduced clickstream data as a source of daily demand signals for an online shop and described the *e-shop clothing 2008* dataset, in which each row is one product view and no purchases are recorded. We have argued that the common `price2` classification task suffers from target leakage and memorisation and has no business value, and we have reformulated the problem as one-step-ahead forecasting of daily product views. The main challenges – temporal leakage, a small count target, short history, confounding by display position, data-quality issues and missing context – motivate the theoretical choices of Chapter 2 and the methodology of Chapter 3.

# CHAPTER 2. THEORETICAL BACKGROUND AND RELATED WORK

This chapter provides the theoretical basis of the project. We first formulate the forecasting of daily product views as a supervised learning problem and define its inputs and outputs. We then review the main families of methods used for demand forecasting, together with the main pitfalls in their evaluation. Finally, we present the theory of every algorithm and metric used in Chapters 3 and 4.

## 2.1 Problem formulation

Let $p \in \{1, \ldots, P\}$ index the products ($P = 217$) and $d$ index the days. The **target** is

$$y_{p,d} = \text{number of clicks on the page of product } p \text{ during day } d, \tag{2.1}$$

a non-negative integer. Let $\mathscr{F}_{d-1}$ denote all information available at the end of day $d-1$: the views of every product up to day $d-1$, the static attributes of the products and the calendar. The task is to learn a function $f$ such that

$$\hat{y}_{p,d} = f(\mathbf{x}_{p,d}), \qquad \mathbf{x}_{p,d} \text{ computed only from } \mathscr{F}_{d-1}, \tag{2.2}$$

where $\mathbf{x}_{p,d}$ is the feature vector of the pair $(p,d)$. Since the output is a number, this is a **supervised regression** problem with a **time-series** structure: the forecast horizon is one day, and a single ("global") model is learned for all products. The requirement that $\mathbf{x}_{p,d}$ uses only $\mathscr{F}_{d-1}$ is the formal definition of the absence of data leakage. Table 2.1 summarises the inputs and outputs.

| Component | Description |
|---|---|
| Unit of observation | One (product, day) pair |
| Input | Views of the product on previous days (lags and rolling means), day of week of the forecast day, static product attributes (category, colour, photo position and type, page, price, `price2`) |
| Output | Forecast of the number of views of the product on the next day |
| Problem type | Supervised regression with time-series features (one-step-ahead forecasting) |
| Main metric | Mean Absolute Error (MAE), complemented by RMSE, MAPE (on $y > 0$) and $R^2$ |
| Key constraint | Time-ordered split; features use only information before the forecast day |

**Table 2.1 Inputs and outputs of the forecasting problem.**

## 2.2 Related work

### *2.2.1 Demand forecasting in retail and e-commerce*

Retail forecasting has been studied for decades; Fildes et al. [3] review the research and practice of forecasting at product, store and aggregate levels and emphasise that most operational decisions rely on simple, robust methods. Ferreira et al. [4] show for an online retailer that product-level demand forecasts can be turned into better pricing decisions. The M5 competition [8], based on daily unit sales of thousands of retail products, found that *global* machine learning models trained on many series at once with lag and rolling-window features were among the most accurate approaches, and that simple benchmarks are often hard to beat by a large margin. Our formulation follows this line: a single model for all products, fed with lag and rolling features.

### *2.2.2 Naive and classical time-series methods*

The simplest forecasts are the *naive* forecast (tomorrow equals today), the *seasonal naive* forecast (tomorrow equals the same day last week) and the *moving average* of recent days [9]. More elaborate statistical models include exponential smoothing and ARIMA models [9, 10]. These methods are usually fitted separately to each series. With 217 short series and many zeros, separate models are fragile; we therefore use the naive, seasonal naive and moving-average forecasts as **baselines** that any machine learning model must beat.

### 2.2.3 Machine learning with lag features

Machine learning regressors can be used for forecasting once the history of each series is turned into features [8, 9]. Linear Regression and its regularised variant Ridge Regression [11] capture additive effects; K-Nearest Neighbours [12] predicts from similar past situations; decision trees [13, 14] and Random Forests [15] capture non-linear interactions such as "a popular product on a Saturday". All these algorithms are implemented in scikit-learn [16], which is used in this project.

### 2.2.4 Classification-based formulations

A forecasting problem can also be turned into a yes/no question, for example "will the product receive at least $\tau$ views tomorrow?". This allows classifiers such as Logistic Regression [17] and Naive Bayes [18] to be used, and evaluation by Precision, Recall and F1 [19]. The price is a loss of information: the exact number of views is replaced by a binary label.

### 2.2.5 Deep learning

Recurrent networks such as LSTM [20] can learn directly from sequences. They usually require more data and tuning than are available in a 134-day log and are harder to explain, so they are outside the scope of this project.

### 2.2.6 Evaluation pitfalls

Three pitfalls recur in the literature. *Leakage* [5]: information that would not be available at prediction time enters the inputs or the preprocessing. *Inappropriate validation* [6]: random splits of time-ordered data let the model learn from the future. *Inappropriate metrics* [7]: percentage errors are undefined for zero actuals and unstable for small ones. Table 2.2 summarises the families of methods and their role in this project.

| Family | Examples | Strengths / weaknesses | Role here |
|---|---|---|---|
| Naive rules | Yesterday, same day last week, 7-/14-day average | Simple, robust; cannot combine several signals | Baselines |
| Statistical time series | Exponential smoothing, ARIMA | Well founded; one model per series, fragile on short sparse series | Not used |
| Linear models | Linear, Ridge Regression | Interpretable coefficients; only additive effects | Regression models |
| Instance-based | KNN | Intuitive; sensitive to scaling and irrelevant features | Regression and classification |
| Tree-based | Decision Tree, Random Forest | Non-linear, no scaling needed; a single tree is unstable | Regression and classification |
| Probabilistic classifiers | Logistic Regression, Naive Bayes | Output class probabilities; only for labels | Classification view |
| Deep learning | LSTM | Learns from raw sequences; data-hungry, less interpretable | Not used |

**Table 2.2 Families of methods for demand forecasting and their role in this project.**

## 2.3 Theoretical foundations

### *2.3.1 Linear Regression*

Linear Regression models the target as a weighted sum of the features,

$$\hat{y}_i = \beta_0 + \sum_{j=1}^{m} \beta_j x_{ij}, \tag{2.3}$$

and chooses the coefficients by *ordinary least squares*, i.e. by minimising the sum of squared residuals

$$\min_{\beta} \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2. \tag{2.4}$$

The coefficient $\beta_j$ is the change in the prediction when $x_{ij}$ increases by one unit and the other features are fixed [21]. Linear Regression can produce negative predictions, which are meaningless for view counts; in this project negative predictions are set to zero before evaluation.

### *2.3.2 Ridge Regression*

When several features carry almost the same information (*multicollinearity*), the least-squares coefficients can become large and unstable. Ridge Regression [11] adds an $L_2$ penalty:

$$\min_{\beta} \sum_{i=1}^{n} \left(y_i - \hat{y}_i\right)^2 + \alpha \sum_{j=1}^{m} \beta_j^2, \tag{2.5}$$

where $\alpha \geq 0$ controls the strength of the penalty. With $\alpha = 0$ Ridge is identical to Linear Regression; as $\alpha$ grows, the coefficients shrink towards zero and are shared more evenly among correlated features. Because the penalty depends on the size of the coefficients, the features must be on a common scale.

### *2.3.3 Standardisation*

Standardisation rescales each numeric column to mean 0 and standard deviation 1:

$$z = \frac{x - \mu_{\text{train}}}{s_{\text{train}}}, \tag{2.6}$$

where $\mu_{\text{train}}$ and $s_{\text{train}}$ are computed **on the training data only** and then reused for the validation and test data; computing them on all data would leak information from the evaluation periods. Standardisation is required by Ridge (penalty) and KNN (distances) and helps Logistic Regression converge; tree-based models are unaffected by it.

### *2.3.4 One-hot encoding*

Categorical codes such as `colour` or `day_of_week` have no numerical order. One-hot encoding replaces a variable with $L$ levels by $L-1$ binary columns. The first level is dropped and serves as the reference: it is represented by all binary columns being zero. Keeping all $L$ columns together with an intercept would make the columns linearly dependent (the *dummy-variable trap*).

### 2.3.5 K-Nearest Neighbours

KNN [12] predicts from the $k$ training points closest to the query point $\mathbf{x}$ in Euclidean distance,

$$\text{dist}(\mathbf{x}, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{m} (x_j - x_{ij})^2}, \qquad \hat{y}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i, \tag{2.7}$$

where $N_k(\mathbf{x})$ is the set of the $k$ nearest neighbours. For classification, the prediction is the majority class among the neighbours. Because every column contributes to the distance, KNN is sensitive both to the scale of the features and to the presence of irrelevant features. A small $k$ gives noisy predictions; a large $k$ over-smooths them.

### 2.3.6 Decision Tree

A regression tree (CART [13]) recursively splits the data with questions of the form "$x_j \leq t$?". At each node it chooses the feature $j$ and threshold $t$ that minimise the weighted squared error of the two child nodes,

$$\min_{j,t} \left[ \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2 \right], \tag{2.8}$$

where $L$ and $R$ are the left and right children and $\bar{y}_L$, $\bar{y}_R$ their means. A leaf predicts the mean target of the training rows it contains. Without limits, a tree can grow until each leaf contains a single row and memorise the training data (*overfitting*); the maximum depth (`max_depth`) and the minimum number of rows per leaf (`min_samples_leaf`) control this. Trees capture non-linear effects and interactions and do not require scaling, but a single tree is unstable and its predictions take only as many distinct values as it has leaves.

### 2.3.7 Random Forest

A Random Forest [15] averages $B$ trees, each trained on a *bootstrap* sample of the training data (rows drawn with replacement):

$$\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}). \tag{2.9}$$

Averaging reduces the variance of individual trees. Classical Random Forests also consider only a random subset of features at each split. Note that in scikit-learn the default for `RandomForestRegressor` is to consider all features (`max_features=1.0`), so the trees of our regression forest differ only through bootstrapping, whereas `RandomForestClassifier` considers $\sqrt{m}$ features by default [16]. Impurity-based feature importances measure how much each feature reduces the error inside the trees; they are biased towards continuous features with many possible split points and towards one of several correlated features [22].

### 2.3.8 *Logistic Regression*

Logistic Regression [17] is a classifier. It computes a linear score and maps it to a probability with the sigmoid function:

$$z = \beta_0 + \sum_j \beta_j x_j, \qquad P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{2.10}$$

and predicts class 1 when this probability is at least 0.5. The coefficients are estimated by minimising the log-loss (equivalently, maximising the likelihood); scikit-learn adds an $L_2$ penalty by default, controlled by the parameter $C$ (a smaller $C$ means a stronger penalty).

### 2.3.9 *Gaussian Naive Bayes*

Naive Bayes applies Bayes' rule with the "naive" assumption that the features are conditionally independent given the class [18]:

$$P(y = c \mid \mathbf{x}) \;\propto\; P(y = c) \prod_{j=1}^{m} P(x_j \mid y = c), \tag{2.11}$$

where, in the Gaussian variant, each $P(x_j \mid y = c)$ is a normal density whose mean and variance are estimated from the training rows of class $c$. When features are strongly dependent—for example several columns encoding the same variable, or two highly correlated averages—the same evidence is counted several times and the predicted probabilities become over-confident.

## 2.4 Evaluation metrics

### 2.4.1 *Regression metrics*

For $n$ observations with actual values $y_i$, predictions $\hat{y}_i$ and mean $\bar{y}$:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|, \tag{2.12}$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}, \tag{2.13}$$

$$\text{MAPE} = \frac{100\%}{|I|} \sum_{i \in I} \frac{|y_i - \hat{y}_i|}{y_i}, \qquad I = \{i : y_i > 0\}, \tag{2.14}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{2.15}$$

MAE is expressed in views and is easy to explain ("on average the forecast is off by about two views"); it is the **main metric** of this project. RMSE penalises large errors more heavily and is always at least as large as MAE. MAPE is computed only on rows with $y_i > 0$ because it is undefined for zero actuals; even then it is inflated by small

actuals (an actual of 1 and a forecast of 2 is a 100% error) [7]. $R^2$ compares the model with always predicting the mean: $R^2 = 1$ is perfect, $R^2 = 0$ is no better than the mean.

**Worked example.** For actual values $(10, 5, 0)$ and predictions $(12, 4, 1)$ the errors are $(2, 1, 1)$, so MAE $= 4/3 = 1.333$, RMSE $= \sqrt{6/3} = 1.414$, MAPE $= (2/10 + 1/5)/2 = 20\%$ (the zero actual is excluded) and, with $\bar{y} = 5$, $R^2 = 1 - 6/50 = 0.88$. The notebook uses this example to test its evaluation function.

### *2.4.2 Classification metrics*

With true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{2.16}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{2.17}$$

When the positive class is rare, Accuracy is misleading: a model that always predicts the majority class can have high Accuracy and zero Recall. F1, the harmonic mean of Precision and Recall, is low whenever either of them is low and is therefore used as the main classification metric [19].

## 2.5 Chapter summary

This chapter has formulated the task as one-step-ahead supervised regression in which every feature must be computed from information available before the forecast day. The literature review has shown that simple rules are strong benchmarks in retail forecasting, that global machine learning models with lag and rolling features are a well-established approach, and that leakage, random validation and percentage metrics are the main pitfalls. We have presented the theory of Linear and Ridge Regression, standardisation and one-hot encoding, KNN, Decision Trees, Random Forests, Logistic Regression and Gaussian Naive Bayes, and the metrics used to compare them. Chapter 3 describes how these elements are combined in the project.

# CHAPTER 3. METHODOLOGY

This chapter describes, step by step, the pipeline implemented in the notebook `DS66B_group_3.ipynb`. Each section corresponds to one part of the notebook (Table 3.1); the variable and function names used below are the same as in the code, so that every statement can be checked directly against it. All numbers in this chapter come from the outputs of the notebook.

| Notebook part | Content | Report section |
|---|---|---|
| Part 1 | Loading the data and first checks | 3.1 |
| Part 2 | Data quality checks and cleaning | 3.2 |
| Part 3 | Re-examination of `price2`; choice of the problem and of the algorithms | 3.3, 4.3 |
| Part 4 | Product–day panel, exploratory analysis, feature engineering | 3.4–3.6 |
| Part 5 | One-hot encoding and time-based split | 3.7 |
| Part 6 | Baselines and models, hyperparameters chosen on validation | 3.8, 4.4–4.7 |
| Part 7 | Refitting on train+validation and single test evaluation | 3.9, 4.8 |
| Part 8 | Result analysis | 3.10, 4.9–4.12 |
| Part 9 | Classification view | 3.11, 4.13 |
| Part 10 | Conclusion | Conclusion |

**Table 3.1 Correspondence between the parts of the notebook and the sections of this report.**

## 3.1 Data loading and first checks

The file is read with `pd.read_csv('e-shop_clothing_2008.csv', sep=';')`, because the columns are separated by semicolons. The 14 columns are renamed to short names in their original order (`year`, `month`, `day`, `order`, `country`, `session`, `category`, `product`, `colour`, `location`, `photo`, `price`, `price2`, `page`). The checks `df.info()`, `df.isnull().sum()` and `df.duplicated().sum()` show that only `product` is a text column and that there are no missing values and no duplicated rows. Duplicates matter particularly here because the target is a count of clicks: a duplicated click would inflate the number of views. Displaying all rows of session 1 (nine rows with `order` from 1 to 9) confirms that one row is one click.

## 3.2 Data quality checks and cleaning

### *3.2.1 Date column and calendar continuity*

The three columns `year`, `month`, `day` are combined into a `date` column with `pd.to_datetime`. The data start on 1 April and end on 13 August 2008; comparing the 135 observed dates with the full calendar produced by `pd.date_range` shows that no day is missing. Continuity matters because the lag features of Section 3.6 take "the previous row" of each product as "yesterday".

### *3.2.2 Truncated last day*

Figure 3.1 plots the total number of clicks per day. The median is 1,203 clicks per day, but the last day (13 August) has only **200 clicks**, about 17% of the median, whereas the four preceding days have between 929 and 1,374 clicks. Because 13 August is the last day of the file, the log was most probably cut during the day. Keeping it would evaluate the models on an artificially low day, so it is removed, leaving **165,274 rows** (134 days). The plot also shows very high traffic on 1–2 April (up to 3,456 clicks on 2 April), which lies before the period used for training (Section 3.6).

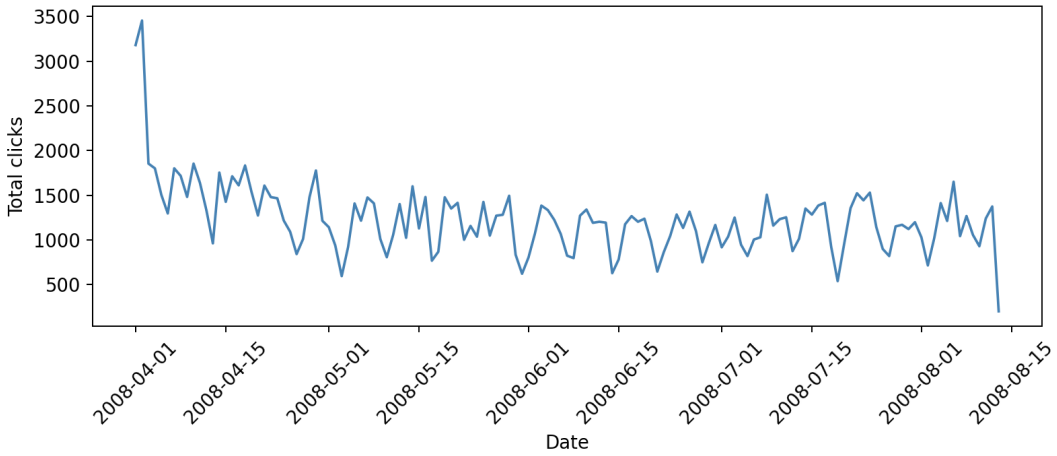



**Figure 3.1 Total number of clicks per day, 1 April–13 August 2008. The last day (13 August) contains only 200 clicks.**

### *3.2.3 Consistency of product attributes and correction of A18*

For each attribute, the notebook counts the products that take more than one value over time. Colour, location, photo, price, `price2` and page are constant for every product; only `category` has one exception, product **A18**, which is recorded 935 times in category 1 and once in category 2. To decide which value is correct, the first letter of the product code is cross-tabulated with the category (Table 3.2). Codes starting with A, B, C and P belong to categories 1, 2, 3 and 4 respectively, without exception except the single click of A18. That click is therefore a recording error, and A18 is set to category

1. After the correction no product has more than one category.

| First letter of code | Category 1 | Category 2 | Category 3 | Category 4 |
|---|---|---|---|---|
| A | 49,685 | 1 | 0 | 0 |
| B | 0 | 38,378 | 0 | 0 |
| C | 0 | 0 | 38,537 | 0 |
| P | 0 | 0 | 0 | 38,673 |

**Table 3.2 Number of clicks by first letter of the product code and recorded category (before correcting A18).**

## 3.3 Design of the `price2` re-examination

To test the common approach discussed in Section 1.3, the notebook defines the label `y_click` $= \mathbb{1}[$`price2` $= 1]$ for every click and trains decision tree classifiers in three settings:

- **Experiment 1:** inputs `category`, `colour`, `location`, `photo`, `price`, `page`; random 80/20 split of the clicks (`train_test_split`, `random_state=42`); `DecisionTreeClassifier(max_depth=5)`.

- **Experiment 2:** the same inputs without `price`; random 80/20 split of the clicks; a decision tree with default settings.

- **Experiment 3:** the same inputs as Experiment 2, but the split is made by *product*. The 217 product codes are sorted alphabetically and, for $i = 0, \ldots, 4$, the codes `products[i::5]` (positions $i$, $i+5$, $i+10$, ...) form the test group, while the model is trained on the clicks of all other products. Each product is therefore tested exactly once, on a model that has never seen it. The mean accuracy over the five groups is compared with the accuracy of always predicting the majority class.

## 3.4 Building the product–day panel

### *3.4.1 Complete grid with zero counts*

The clicks are aggregated into one row per product and day. A cross join of the 217 products with the 134 days (`pd.merge(..., how='cross')`) produces the complete grid of **29,078** pairs. The number of clicks per pair (`df.groupby(['product','date']).size()`) is available for only 26,127 pairs; the other pairs had no clicks and receive `views` $= 0$. These zeros are true zeros (nobody viewed the product that day), not missing values. Keeping them is also essential for the lag features: if days without clicks were dropped, "the previous row" of a product would

no longer be “yesterday”. The static attributes of each product are then merged from the 217-row table `catalog`.

### *3.4.2 Product life cycle*

The panel assumes that every product is on sale during all 134 days. Checking the last day on which each product was clicked shows that four products—**P22, P28, P54 and P79**—have no clicks after June; they were clicked on only 2, 1, 1 and 2 days respectively, all in April. They are kept, because their zeros are real, but they are “easy” rows, so Section 4.10 checks that the conclusions do not depend on them.

## 3.5 Exploratory analysis of the target

The exploratory analysis uses **only the training period (up to 15 July)**; the validation and test periods are kept unseen, so that decisions about features cannot be influenced by them.

### *3.5.1 Distribution.*

Over the 23,002 product–day pairs up to 15 July, `views` has median 4, mean 5.77, standard deviation 5.97, quartiles 2 and 8 and maximum 79; **10.2%** of the pairs are zeros (Figure 3.2). The target is a right-skewed count, which motivates computing MAPE only on positive actuals and clipping negative predictions to zero.

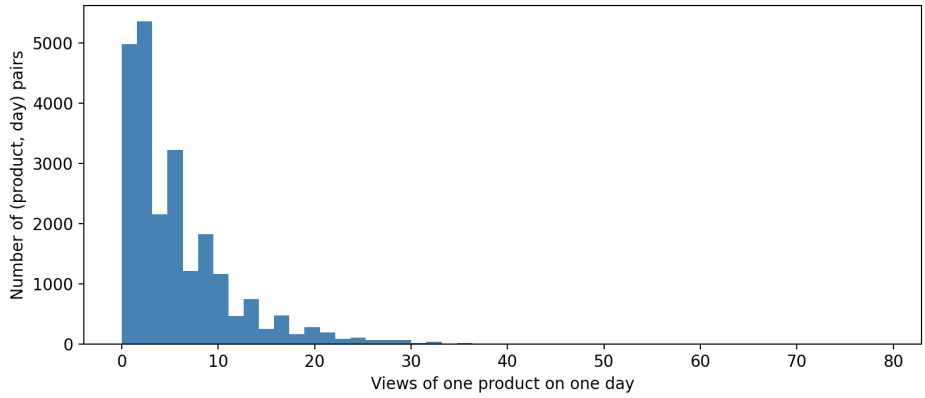



**Figure 3.2 Distribution of the target `views` (product–day pairs up to 15 July).**

### *3.5.2 Weekly cycle.*

The average number of views per product is highest on Tuesday and Wednesday (6.75 and 6.81) and lowest on Saturday (4.17) and Sunday (4.62) (Figure 3.3). This clear weekly pattern motivates the feature `day_of_week` and the lag of seven days.

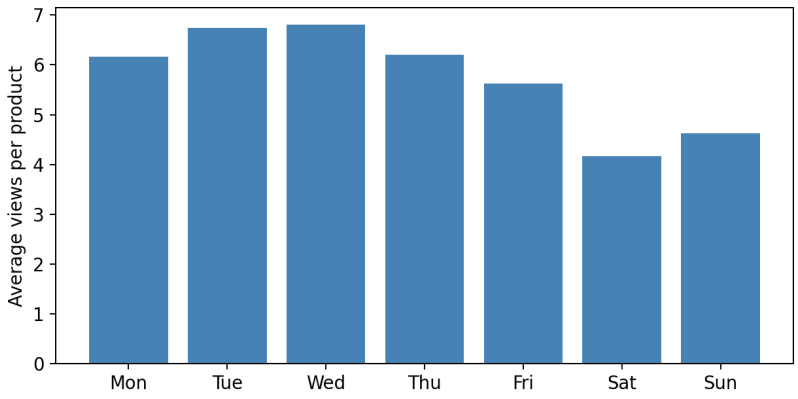



**Figure 3.3 Average views of one product by day of the week (training period).**

### *3.5.3 Category and page.*

Average daily views per product are 9.11 for skirts, 8.73 for trousers, 4.99 for blouses and 3.44 for sale items (Figure 3.4), and they fall from 9.92 on page 1 to 1.87 on page 5 (Figure 3.5). Table 3.3, however, shows that all skirts are on pages 1–2 while sale items are spread over all five pages. Differences between categories are therefore *confounded* with display position, and no causal conclusion (e.g. "putting a product on sale reduces its views") can be drawn. Category and page are nevertheless useful inputs for prediction.

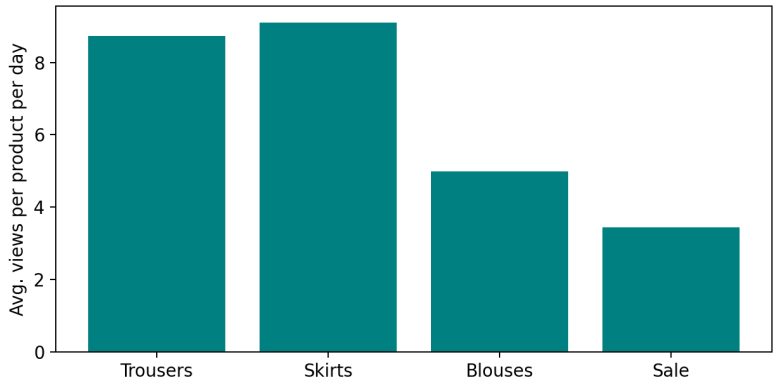



**Figure 3.4 Average views per product per day by main category ("Sale" is the closest proxy for promotions).**

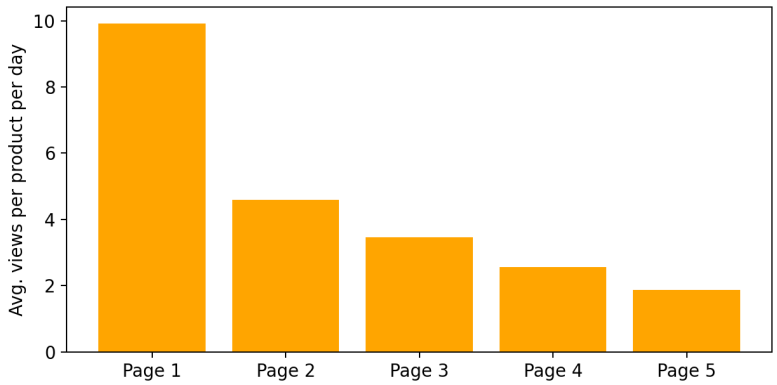



**Figure 3.5 Average views per product per day by page of the website.**

| Category | Page 1 | Page 2 | Page 3 | Page 4 | Page 5 |
|---|---|---|---|---|---|
| Blouses | 18 | 18 | 18 | 5 | 0 |
| Sale | 18 | 18 | 17 | 19 | 10 |
| Skirts | 17 | 16 | 0 | 0 | 0 |
| Trousers | 18 | 18 | 7 | 0 | 0 |

**Table 3.3 Number of products of each category on each page.**

## 3.6 Feature engineering and leakage control

### *3.6.1 Features*

To forecast day $d$, every history feature must use only days up to $d - 1$. All computations are done **within each product** (`grp = panel.groupby('product')['views']`), so that the history of one product never leaks into another. Table 3.4 lists the features.

| Feature | Definition | Code |
|---|---|---|
| `lag_1` | $y_{p,d-1}$: views yesterday | `grp.shift(1)` |
| `lag_7` | $y_{p,d-7}$: views on the same weekday last week | `grp.shift(7)` |
| `avg_7_days` | $\frac{1}{7}\sum_{l=1}^{7} y_{p,d-l}$ | `grp.transform( avg_last_7_days)` |
| `avg_14_days` | $\frac{1}{14}\sum_{l=1}^{14} y_{p,d-l}$ | `grp.transform( avg_last_14_days)` |
| `day_of_week` | Day of the week of day $d$ (0 = Monday) | `panel['date'].dt.dayofweek` |
| Static attributes | `category`, `colour`, `location`, `photo`, `page`, `price2`, `price` | merged from `catalog` |

**Table 3.4 Features used to forecast the views of product $p$ on day $d$.**

The rolling means are computed by two small named functions that always shift by one day *before* averaging:

```
def avg_last_7_days(x):
    return x.shift(1).rolling(7).mean()

def avg_last_14_days(x):
    return x.shift(1).rolling(14).mean()
```

Writing `x.rolling(7).mean()` without the shift would include the views of day $d$ itself, i.e. the answer to be predicted. Section 4.12 measures the effect of this mistake. The feature `day_of_week` refers to the forecast day itself, but this is not leakage: the calendar is known in advance.

### *3.6.2 Manual leakage check*

For product A1 on 20 April, the views from 13 to 19 April are $(12, 27, 19, 23, 16, 17, 11)$, whose mean is 17.857. The value of `avg_7_days` computed by the notebook for 20 April is also 17.857, and `lag_1` equals 11, the actual number of views on 19 April. The features of 20 April therefore do not use the views of 20 April.

### *3.6.3 Warm-up period*

The first 14 days (1–14 April) do not have enough history for `avg_14_days` and are removed with `panel.dropna(subset=history_cols)`, where `history_cols` lists

the four history features. Restricting `dropna` to these columns prevents rows from being removed because of any other column. The modelling table `panel_clean` contains **26,040 rows** (217 products $\times$ 120 days, 15 April–12 August).

## 3.7 Encoding and time-based split

The seven categorical columns (`category`, `colour`, `location`, `photo`, `page`, `price2`, `day_of_week`) are one-hot encoded with `pd.get_dummies(..., drop_first=True)`. Together with the five numeric columns (`price`, `lag_1`, `lag_7`, `avg_7_days`, `avg_14_days`) this gives a feature matrix `X_all` with **38 columns** (5 numeric and 33 binary). One-hot encoding learns no statistic from the data, so applying it before the split does not cause leakage.

The data are split by date, never at random (Table 3.5). The training period is used to fit models, the validation period to choose hyperparameters and models, and the test period only once for the final evaluation.

| Set | From | To | Rows |
|---|---|---|---|
| Train | 15 April 2008 | 15 July 2008 | 19,964 |
| Validation | 16 July 2008 | 31 July 2008 | 3,472 |
| Test | 1 August 2008 | 12 August 2008 | 2,604 |

**Table 3.5 Time-based split of the modelling table.**

For the linear models the features are standardised with a `StandardScaler` fitted on the training set only (`scaler_train.fit_transform(X_train)`) and then applied to the validation set (`scaler_train.transform(X_val)`).

## 3.8 Models and hyperparameter selection

All predictions are evaluated by the function `evaluate`, which clips negative predictions to zero (`np.maximum(y_pred, 0)`), computes MAE, RMSE, MAPE on $y > 0$ and $R^2$, and was verified on the worked example of Section 2.4. Table 3.6 lists the models, their inputs and the hyperparameter grids. For every grid, each value is trained on the training set and scored by MAE on the validation set, and the value with the lowest MAE is selected automatically (`np.argmin` or sorting).

| Model | Inputs | Scaling | Hyperparameters |
|---|---|---|---|
| 0a–0d. Baselines | `lag_1`, `lag_7`, `avg_7_days`, `avg_14_days` used directly as forecasts | — | none |
| 1a. Linear Regression | `lag_1` only | — | none |
| 1b. Linear Regression | all 38 columns | standardised | none |
| 2. Ridge Regression | all 38 columns | standardised | $\alpha$: 0.1, 1, 10, 100, 1000, 10000 |
| 3. KNN regressor | three feature sets: all 38 columns; 4 history columns; 4 history columns + 6 day-of-week columns | standardised (scaler per feature set) | $k$: 10, 30, 50, 100, 200 |
| 4. Decision Tree | all 38 columns | — | `max_depth`: 3, 5, 6, 7, 10, 12, 15; `min_samples_leaf` = 20 |
| 5. Random Forest | all 38 columns | — | `max_depth`: 6, 8, 10, 12; 100 trees; `min_samples_leaf` = 20 |

**Table 3.6 Models, inputs and hyperparameter grids (all with `random_state=42` where applicable).**

The four baselines require no training: "yesterday" (0a), "same day last week" (0b), "7-day average" (0c) and "14-day average" (0d). Model 1a shows what a learned model can extract from yesterday's value alone. For KNN, the feature set is tuned together with $k$, because irrelevant columns distort the distances; the 6 day-of-week columns are `day_of_week_1` to `day_of_week_6`.

## 3.9 Final evaluation protocol

After all hyperparameters have been chosen, the model with the lowest validation MAE is selected. All models are then refitted on the training and validation periods together (`dev_mask`: up to 31 July, 23,436 rows), with the scalers refitted on the same

data, and evaluated **once** on the test period. Refitting adds the 16 most recent days, which are closest to the test period. The procedure is summarised in Algorithm 1.

**Algorithm 1** Model selection and evaluation protocol

**for** each model $M$ and each value $h$ of its hyperparameter grid **do**
    Fit $M_h$ on Train; compute MAE of $M_h$ on Validation
**end for**
For each model, keep $h^* = \arg\min_h \mathrm{MAE}_{\mathrm{val}}(M_h)$
Select the model with the lowest validation MAE; select the strongest baseline on validation
Refit every model with $h^*$ on Train $\cup$ Validation (scalers refitted as well)
Evaluate every model once on Test; report the improvement of the selected model over the strongest baseline

## 3.10 Analysis protocol

The test results are analysed in five ways:

- **Strongest baseline.** The improvement is measured against the baseline with the lowest *validation* MAE, found by a simple loop over the four baselines, so that the reference is not chosen after seeing the test results.

- **Day-by-day reliability.** For each test day the MAE of the selected model is compared with that of the strongest baseline, and the number of days won is counted. If the two methods were equally good, each day would be like a fair coin flip. The notebook simulates 12 coin flips 10,000 times (`np.random.seed(42)`) and records the proportion of simulations with at least as many wins as the model; this proportion is a simulated one-sided $p$-value.

- **Error analysis.** Actual and predicted values are compared per product–day and per day, and the MAE and the bias (mean of prediction minus actual) are computed by category. The MAE is recomputed without the four inactive products of Section 3.4.

- **Feature importance.** The impurity-based importances of the final Random Forest are reported, with the caveats of Section 2.3.

- **Leakage stress test.** The 7-day average is deliberately recomputed without `shift(1)` (function `avg_7_days_WRONG`, with `min_periods=1`), and the same Random Forest is retrained and tested, to measure how much a leak would inflate the scores.

## 3.11 Classification reformulation

Logistic Regression and Naive Bayes are classifiers and cannot forecast a number of views. To use them appropriately and to test a different framing of the problem, the notebook defines a binary question: *will the product be of high interest tomorrow?* The label is 1 if `views` $\geq \tau$, where $\tau$ is the 75th percentile of the training target (`y_train.quantile(0.75)`), which equals **7 views**. Computing $\tau$ on the training set only avoids leakage. The proportion of positive labels is 29.4% in training, 28.7% in validation and 28.8% in test.

Eight approaches are compared: (0) a rule, "14-day average $\geq \tau$"; (R) the Random Forest *regressor* of Part 6 followed by the same threshold; (1) Logistic Regression (`max_iter=1000`) on the standardised 38 columns; (2a) Gaussian Naive Bayes on the 38 columns and (2b) on the 4 history columns; (3) a KNN classifier with the feature set and $k$ selected in Part 6; (4) a Decision Tree classifier and (5) a Random Forest classifier with the depths selected in Part 6 and `min_samples_leaf`$= 20$. Labels are obtained with `.predict()`, i.e. a probability threshold of 0.5 (or a majority vote for KNN). Because the classes are imbalanced, F1 is the selection metric; the approach with the highest validation F1 is selected, refitted on train+validation and evaluated once on the test set.

## 3.12 Chapter summary

This chapter has described the complete methodology: loading and cleaning the data (removing the truncated last day and correcting product A18), designing three experiments to re-examine the `price2` task, building a 29,078-row product–day panel with true zeros, analysing the target on the training period only, engineering past-only features within each product and checking them manually, encoding and splitting the data by time, tuning four baselines and six regression configurations on validation, evaluating once on test and analysing the results, and finally reformulating the task as a classification problem. Chapter 4 presents the results obtained with this methodology.

# CHAPTER 4. EXPERIMENTS AND EVALUATION

This chapter reports the results of all experiments described in Chapter 3. Every number in the tables and figures is produced by the notebook `DS66B_group_3.ipynb`; model names keep the identifiers used in the code (0a–0d for baselines, 1a–5 for models) so that each row can be matched with the notebook output.

## 4.1 Experimental objectives

The experiments aim to answer the following questions:

- Does the common `price2` classification task measure genuine predictive ability? (Section 4.3)
- How well do simple rules forecast daily product views, and which one is the strongest? (Section 4.4)
- Which hyperparameters and feature sets work best for each algorithm, and which model is best on validation? (Sections 4.4–4.7)
- How large is the improvement on the unseen test period, and is it reliable? (Sections 4.8–4.9)
- Where does the model make errors, which features does it rely on, and how much would a leak inflate the scores? (Sections 4.10–4.12)
- Does a classification formulation offer any advantage? (Section 4.13)

## 4.2 Experimental setup

### *4.2.1 Computing environment*

The results in this report were obtained by running the notebook from top to bottom in a fresh session with Python 3.12, pandas 3.0.2 [23], NumPy 2.4.4 [24], scikit-learn 1.8.0 [16] and Matplotlib 3.10.8 [25]. All random procedures use `random_state=42` (or `np.random.seed(42)`), so the results are reproducible; running the notebook with other library versions may change the last decimal of some metrics.

### *4.2.2 Dataset*

Table 4.1 summarises the data at each stage of the pipeline.

| Stage | Size |
|---|---|
| Raw clicks (1 April–13 August 2008) | 165,474 rows × 14 columns |
| After removing the truncated day 13 August | 165,274 rows (134 days) |
| Products / sessions | 217 / 24,026 |
| Product–day panel (complete grid) | 29,078 rows (26,127 pairs with at least one click) |
| Modelling table after the 14-day warm-up | 26,040 rows (15 April–12 August) |
| Feature matrix after one-hot encoding | 38 columns |
| Train / validation / test | 19,964 / 3,472 / 2,604 rows |

**Table 4.1 Size of the data at each stage.**

## 4.3 Experiment 0: re-examining the `price2` task

Table 4.2 reports the three experiments designed in Section 3.3.

| Experiment | Setting | Accuracy |
|---|---|---|
| 1 | With `price`, random split of clicks | 100.00% |
| 2 | Without `price`, random split of clicks | 92.91% |
| 3 | Without `price`, tested on unseen products (5 product groups: 27.1, 45.1, 46.4, 43.9, 57.8%) | 44.1% (mean) |
| — | Always predicting the majority class | 51.2% |

**Table 4.2 Accuracy of decision trees predicting `price2` under three settings.**

The 100% accuracy of Experiment 1 is explained by target leakage: `price2` is computed from `price` and the category. Removing `price` still gives 92.91% under a random split, but this is memorisation: because product attributes are constant, a combination of attributes identifies a product, and the same products appear in both training and test data. When the products in the test group have never been seen (Experiment 3), accuracy drops to 44.1% on average, below the 51.2% of the trivial majority-class rule; the result varies strongly across the five groups (from 27.1% to 57.8%) because each group contains only about 43 products. The attributes therefore carry no transferable information about `price2`, and the high scores reported for this task do not measure real predictive ability. This supports the reformulation adopted in this report.

## 4.4 Baselines and linear models

### *4.4.1 Baselines*

On the validation period, the four baselines obtain MAE values of 2.666 (yesterday, 0a), 2.648 (same day last week, 0b), 2.182 (7-day average, 0c) and **2.125 (14-day average, 0d)**. Rules based on a single day are much worse than averages, because the views of a single day contain a lot of random variation that averaging removes. The **14-day average is the strongest baseline** and is the reference that the models must beat.

### *4.4.2 Linear Regression*

Trained on `lag_1` alone, Linear Regression learns

$$\widehat{\texttt{views}} = 1.434 + 0.730 \times \texttt{lag_1}, \tag{4.1}$$

with a validation MAE of 2.563. The slope below 1 pulls the forecast towards the average (a product that was unusually popular yesterday is expected to be somewhat less popular today); the model beats the "yesterday" rule but remains far behind the 14-day average. With all 38 standardised columns, Linear Regression reaches a validation MAE of **1.999**, the first configuration that beats the strongest baseline. Feature engineering is therefore essential.

### *4.4.3 Ridge Regression*

Table 4.3 shows the validation MAE of Ridge Regression for six values of $\alpha$.

| $\alpha$ | 0.1 | 1 | 10 | 100 | 1000 | 10000 |
|---|---|---|---|---|---|---|
| Validation MAE | 1.999 | 1.999 | 1.999 | 2.001 | 2.013 | 2.108 |

**Table 4.3 Validation MAE of Ridge Regression for different values of $\alpha$.**

The MAE is practically constant for $\alpha$ between 0.1 and 10 (1.999) and increases only with strong penalties ($\alpha \geq 1000$): the model is insensitive to regularisation in this range, and the selected value $\alpha = 0.1$ (the first of three tied values) gives the same results as Linear Regression. With almost 20,000 training rows and 38 columns, ordinary least squares does not overfit, so the penalty brings no gain in accuracy. Note also that Ridge minimises squared errors while $\alpha$ is selected by MAE; with a right-skewed target the two criteria could in principle prefer different values.

The value of Ridge lies elsewhere. Table 4.4 shows that the history features are strongly correlated, in particular `avg_7_days` and `avg_14_days` (0.983). Table 4.5 compares the coefficients (on standardised features) of Linear Regression and Ridge. With $\alpha = 0.1$ they are identical; with a strong penalty ($\alpha = 1000$) Ridge spreads the weight more evenly across the correlated features—the coefficient of `avg_14_days` falls

from 2.696 to 1.799 while the others increase—at the cost of a slightly higher validation MAE (2.013). In this dataset all coefficients are positive and stable, which explains why regularisation is not needed.

| | lag_1 | lag_7 | avg_7_days | avg_14_days |
|---|---|---|---|---|
| lag_1 | 1.000 | 0.720 | 0.861 | 0.846 |
| lag_7 | 0.720 | 1.000 | 0.861 | 0.851 |
| avg_7_days | 0.861 | 0.861 | 1.000 | 0.983 |
| avg_14_days | 0.846 | 0.851 | 0.983 | 1.000 |

**Table 4.4 Correlation between the history features (training set).**

| **Feature** | **Linear Regression** | **Ridge** ($\alpha = 0.1$) | **Ridge** ($\alpha = 1000$) |
|---|---|---|---|
| lag_1 | 0.493 | 0.493 | 0.613 |
| lag_7 | 0.207 | 0.207 | 0.410 |
| avg_7_days | 0.811 | 0.811 | 1.277 |
| avg_14_days | 2.696 | 2.696 | 1.799 |

**Table 4.5 Coefficients of the history features (standardised inputs) for Linear Regression and Ridge Regression.**

## 4.5 K-Nearest Neighbours

Table 4.6 reports the validation MAE of the KNN regressor for three feature sets and five values of $k$.

| **Feature set** | $k = 10$ | $k = 30$ | $k = 50$ | $k = 100$ | $k = 200$ |
|---|---|---|---|---|---|
| All 38 columns | 2.382 | 2.337 | 2.329 | 2.338 | 2.412 |
| 4 history columns | 2.152 | 2.100 | 2.088 | 2.083 | 2.087 |
| 4 history columns + day of week | 2.072 | 2.002 | **1.985** | 1.996 | 2.019 |

**Table 4.6 Validation MAE of the KNN regressor by feature set and number of neighbours $k$.**

With all 38 columns, KNN performs worst (best MAE 2.329) and is even worse than the 14-day average: the 27 binary columns describing category, colour, photo position, photo type, page and price level (together with price) contribute to the distance without being informative about tomorrow's views, so the "nearest" neighbours are similar in appearance rather than in popularity. Restricting the inputs to the 4 history columns

reduces the MAE to 2.083, and adding the 6 day-of-week columns gives the best result, **1.985 with** $k = 50$, the second-best validation score of all models. Small values of $k$ give noisy averages and large values over-smooth them; the selected $k = 50$ lies inside the grid. For distance-based methods, selecting relevant features is as important as choosing the algorithm.

## 4.6 Decision Tree and Random Forest

Figure 4.1 and Table 4.7 show the effect of the tree depth. The training MAE decreases steadily as the tree grows (from 2.210 to 1.899), whereas the validation MAE reaches its minimum at `max_depth`= 7 (1.992) and then increases again (2.070 at depth 15): deeper trees start to memorise the training data, which is the typical signature of **overfitting**. The best depth is computed automatically from the validation errors.

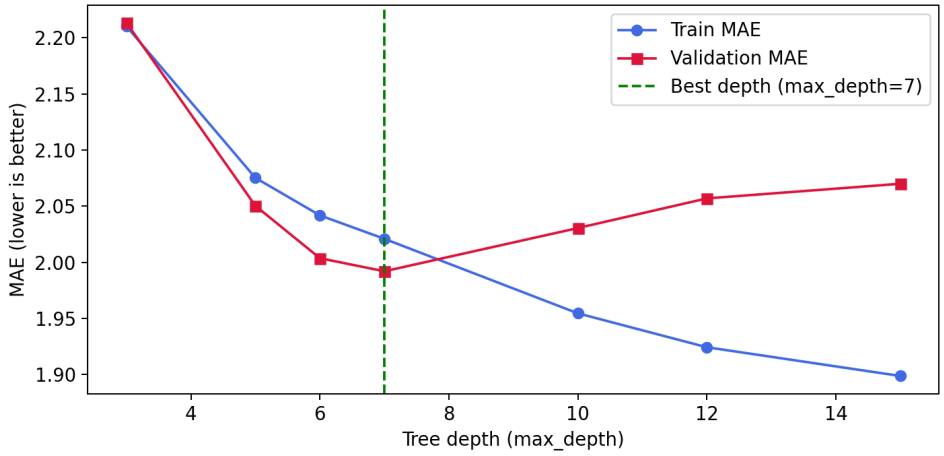



**Figure 4.1 Training and validation MAE of the Decision Tree as a function of `max_depth` (`min_samples_leaf`= 20).**

| `max_depth` | 3 | 5 | 6 | 7 | 10 | 12 | 15 |
|---|---|---|---|---|---|---|---|
| Train MAE | 2.210 | 2.075 | 2.042 | 2.021 | 1.954 | 1.924 | 1.899 |
| Validation MAE | 2.213 | 2.050 | 2.004 | **1.992** | 2.030 | 2.057 | 2.070 |

**Table 4.7 Training and validation MAE of the Decision Tree for each depth.**

For the Random Forest (100 trees, `min_samples_leaf`= 20), the validation MAE is lowest at `max_depth`= 8 (Table 4.8); this value lies inside the grid rather than at its edge.

| max_depth | 6 | 8 | 10 | 12 |
|---|---|---|---|---|
| Validation MAE | 1.980 | **1.963** | 1.965 | 1.968 |

**Table 4.8 Validation MAE of the Random Forest for each depth.**

## 4.7 Model comparison on the validation set

Table 4.9 gathers the validation results of all configurations. All five machine learning models that use the engineered features beat all four baselines; Linear Regression with `lag_1` only does not. The **Random Forest has the lowest validation MAE (1.963)** and is selected; the test set has not been used at this point. The differences between the machine learning models are small (1.963–1.999), and even the gap between the strongest baseline and the best model (2.125 to 1.963) is smaller than the gain obtained simply by replacing "yesterday" with the "14-day average" (2.666 to 2.125). Most of the accuracy therefore comes from using the history correctly rather than from model complexity.

| **Model** | **MAE** | **RMSE** | **MAPE (%)** | **$R^2$** |
|---|---|---|---|---|
| 5. Random Forest (100 trees, `max_depth`= 8) | **1.963** | 2.787 | 51.6 | 0.741 |
| 3. KNN ($k = 50$, history + day of week) | 1.985 | 2.799 | 52.8 | 0.739 |
| 4. Decision Tree (`max_depth`= 7) | 1.992 | 2.845 | 52.0 | 0.730 |
| 1b. Linear Regression (all features) | 1.999 | 2.833 | 55.2 | 0.733 |
| 2. Ridge Regression ($\alpha = 0.1$) | 1.999 | 2.833 | 55.2 | 0.733 |
| 0d. Baseline: 14-day average | 2.125 | 3.046 | 56.5 | 0.691 |
| 0c. Baseline: 7-day average | 2.182 | 3.105 | 58.9 | 0.679 |
| 1a. Linear Regression (`lag_1` only) | 2.563 | 3.532 | 69.3 | 0.584 |
| 0b. Baseline: same day last week (`lag_7`) | 2.648 | 3.752 | 71.0 | 0.531 |
| 0a. Baseline: yesterday (`lag_1`) | 2.666 | 3.744 | 70.2 | 0.533 |

**Table 4.9 Validation results of all baselines and models, sorted by MAE (best MAE in bold).**

## 4.8 Final results on the test set

Table 4.10 reports the test results after refitting every model on the training and validation periods. The five machine learning models keep exactly the same ranking as on validation (Random Forest, KNN, Decision Tree, Linear Regression and Ridge); only the two weakest approaches (0b and 1a) swap places. The Random Forest obtains a test MAE of **2.023** and $R^2 = 0.711$. The strongest baseline, selected on validation, is the 14-day average (test MAE 2.126, $R^2 = 0.686$), so the Random Forest **reduces the error by 4.8%** ($1 - 2.023/2.126$). Against the 7-day average the reduction would be 6.7%, but

we report the improvement over the strongest baseline. The test MAE is slightly higher than the validation MAE (2.023 versus 1.963), which is expected for a completely new period. MAPE values around 50% look high but are dominated by the many products with only 1–3 views per day, for which an error of one view is already 33–100%; this is why MAE is the main metric.

| Model | Val MAE | Test MAE | Test RMSE | Test MAPE (%) | Test $R^2$ |
|---|---|---|---|---|---|
| 5. Random Forest (100 trees, `max_depth`= 8) | 1.963 | **2.023** | 2.938 | 50.5 | 0.711 |
| 3. KNN ($k = 50$, history + day of week) | 1.985 | 2.043 | 2.988 | 50.8 | 0.702 |
| 4. Decision Tree (`max_depth`= 7) | 1.992 | 2.053 | 2.990 | 51.0 | 0.701 |
| 1b. Linear Regression (all features) | 1.999 | 2.078 | 2.991 | 53.9 | 0.701 |
| 2. Ridge Regression ($\alpha = 0.1$) | 1.999 | 2.078 | 2.991 | 53.9 | 0.701 |
| 0d. Baseline: 14-day average | 2.125 | 2.126 | 3.065 | 54.1 | 0.686 |
| 0c. Baseline: 7-day average | 2.182 | 2.168 | 3.114 | 54.8 | 0.676 |
| 1a. Linear Regression (`lag_1` only) | 2.563 | 2.642 | 3.724 | 69.4 | 0.536 |
| 0b. Baseline: same day last week (`lag_7`) | 2.648 | 2.627 | 3.779 | 67.7 | 0.522 |
| 0a. Baseline: yesterday (`lag_1`) | 2.666 | 2.740 | 3.986 | 70.9 | 0.469 |

**Table 4.10 Test results (1–12 August 2008) after refitting on train+validation, sorted by validation MAE (best test MAE in bold).**

## 4.9 Reliability of the improvement

An average MAE may be driven by a few days. Table 4.11 compares, for each test day, the MAE of the Random Forest with that of the 14-day average.

| Date | MAE (Random Forest) | MAE (14-day average) | Model wins? |
|---|---|---|---|
| 2008-08-01 | 1.887 | 1.889 | Yes |
| 2008-08-02 | 1.650 | 2.424 | Yes |
| 2008-08-03 | 1.786 | 1.935 | Yes |
| 2008-08-04 | 2.190 | 2.183 | No |
| 2008-08-05 | 2.006 | 2.001 | No |
| 2008-08-06 | 2.513 | 2.781 | Yes |
| 2008-08-07 | 1.919 | 1.787 | No |
| 2008-08-08 | 2.151 | 2.121 | No |
| 2008-08-09 | 2.182 | 2.122 | No |
| 2008-08-10 | 1.720 | 1.911 | Yes |
| 2008-08-11 | 2.041 | 2.108 | Yes |
| 2008-08-12 | 2.232 | 2.250 | Yes |

**Table 4.11 Daily MAE of the Random Forest and of the strongest baseline on the test period.**

The Random Forest wins on **7 of the 12 days**; it loses on 4, 5, 7, 8 and 9 August, always by small margins, and its largest win is on 2 August (1.650 versus 2.424). If the two methods were equally good, each day would be a fair coin flip. In the notebook's simulation of 10,000 sequences of 12 coin flips, **39.1%** of the sequences contained at least 7 wins. A result like ours would therefore be quite common by pure chance; the conventional threshold for "unlikely by chance" is 5%. Analytically, the probability of at least 10 wins out of 12 is $\left[\binom{12}{10}+\binom{12}{11}+\binom{12}{12}\right]/2^{12} = 79/4096 \approx 1.9\%$, so at least 10 wins would be needed to pass that threshold. The honest conclusion is that the Random Forest **shows potential to improve the forecasts**—its MAE is lower on both validation and test—**but the 12-day test period does not provide sufficient evidence to claim with confidence that it is better than the 14-day average**.

## 4.10 Error analysis

Figure 4.2 compares actual and predicted views for every product–day in the test period. For high actual values most points lie below the diagonal: like any model that learns averages, the Random Forest pulls forecasts towards typical values and under-predicts unusually popular days. Figure 4.3 shows the total views of the shop per day. The forecasts follow the general level but miss sudden spikes such as 6 August (1,651 actual views against 1,220 predicted): no feature announces such shocks, which are likely caused by events outside the data (promotions, campaigns).

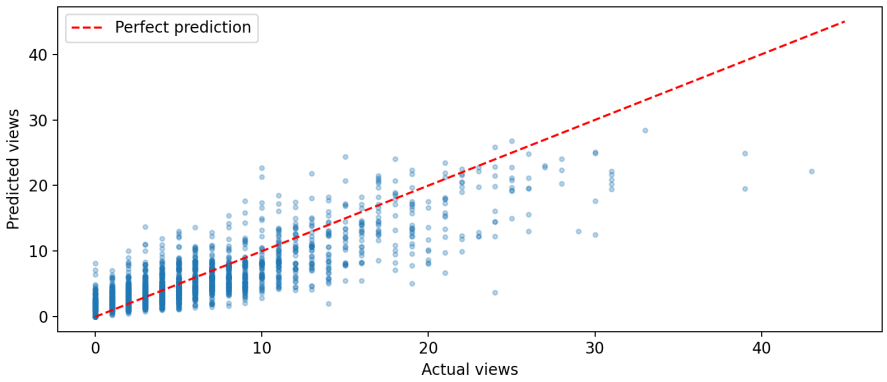



**Figure 4.2 Actual versus predicted views on the test set (Random Forest); each point is one product on one day.**

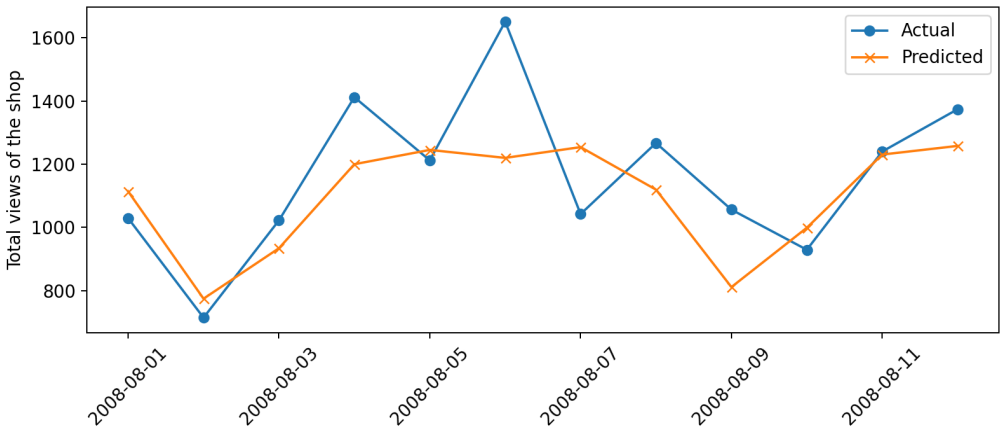



**Figure 4.3 Total daily views of the shop on the test period: actual and predicted (Random Forest).**

Table 4.12 breaks the test error down by category. The MAE is largest for the most viewed categories (trousers and skirts), which is natural because larger counts produce larger absolute errors. The bias is negative in **all four categories**: in August the model under-forecasts slightly (on average 5.05 predicted versus 5.36 actual views per product per day), but no category is affected much more than the others.

| Category | Mean actual views | MAE | Bias (pred.−actual) |
|---|---|---|---|
| Trousers | 7.882 | 2.546 | -0.460 |
| Skirts | 6.992 | 2.526 | -0.261 |
| Blouses | 4.527 | 1.990 | -0.352 |
| Sale | 3.971 | 1.570 | -0.203 |

**Table 4.12 Test error of the Random Forest by category (negative bias = under-forecast).**

**Robustness to inactive products.** Excluding the four products without clicks after June (P22, P28, P54, P79), the test MAE becomes 2.061 for the Random Forest and 2.166 for the 14-day average, against 2.023 and 2.126 with all products. Both errors increase slightly, because the removed rows are easy zeros, but the improvement remains **4.8%**; the conclusion does not depend on these products.

## 4.11 Feature importance

Figure 4.4 shows the ten largest impurity-based importances of the final Random Forest. `avg_14_days` dominates (0.893), followed by `avg_7_days` (0.040) and `day_of_week_5` (Saturday, 0.031), `day_of_week_6` (Sunday, 0.013), `lag_1` (0.008) and `lag_7` (0.004). The Random Forest thus relies mainly on the level of interest over the

last two weeks, consistent with the 14-day average being the strongest baseline, and on the weekly cycle observed in Section 3.5. Three caveats apply [22]: correlated features share importance, so `avg_7_days` appears less important than it is; one-hot encoding splits `day_of_week` into several small columns; and importance shows what the model *uses*, not what *causes* views.

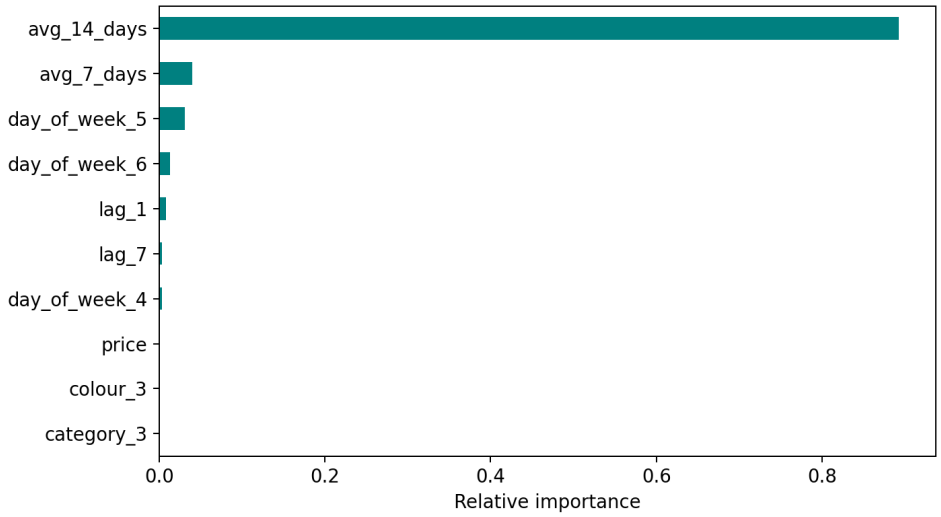



**Figure 4.4 Ten most important features of the Random Forest (impurity-based importance).**

## 4.12 Leakage stress test

Table 4.13 compares the correct Random Forest with the same model trained after recomputing `avg_7_days` without `shift(1)`. The leaky version appears much better (MAE 1.739, $R^2 = 0.786$), but this score is fake: at the time of forecasting, nobody knows tomorrow's views. A model evaluated in this way would perform worse in deployment than its report suggests, which illustrates why the shift and the manual check of Section 3.6 are necessary.

| Random Forest | MAE | RMSE | MAPE (%) | $R^2$ |
|---|---|---|---|---|
| Correct (with `shift(1)`) | 2.023 | 2.938 | 50.5 | 0.711 |
| Leaky (without `shift(1)`) | 1.739 | 2.530 | 43.5 | 0.786 |

**Table 4.13 Effect of forgetting the one-day shift on the test scores.**

## 4.13 Classification view

With the threshold $\tau = 7$ views, about 29% of the labels are positive. Always predicting "not high interest" would give 71.3% accuracy on validation with an F1 of zero, which confirms that Accuracy is not a suitable selection metric here. Table 4.14

reports the validation results.

| Approach | Accuracy | Precision | Recall | F1 |
|---|---|---|---|---|
| 4. Decision Tree (`max_depth`= 7) | 0.863 | 0.805 | 0.687 | **0.741** |
| R. Random Forest regressor + threshold | 0.861 | 0.814 | 0.668 | 0.734 |
| 3. KNN ($k = 50$) | 0.854 | 0.809 | 0.641 | 0.715 |
| 5. Random Forest (`max_depth`= 8) | 0.854 | 0.812 | 0.639 | 0.715 |
| 1. Logistic Regression | 0.855 | 0.828 | 0.623 | 0.711 |
| 0. Rule: 14-day average $\geq$ threshold | 0.839 | 0.762 | 0.639 | 0.695 |
| 2b. Gaussian Naive Bayes (4 history columns) | 0.842 | 0.776 | 0.629 | 0.695 |
| 2a. Gaussian Naive Bayes (38 columns) | 0.821 | 0.779 | 0.526 | 0.628 |

**Table 4.14 Validation results of the classification formulation, sorted by F1 (selected approach in bold).**

On validation, the best learned approaches improve over the rule ($F1 = 0.695$) by only 0.02–0.05, and Naive Bayes does not improve on it at all. The Decision Tree classifier has the highest F1 (0.741) and is selected. The Random Forest *regressor* followed by the threshold (0.734) is almost as good, showing that the regression model already contains the information needed for the yes/no question. Logistic Regression has the highest Precision (0.828) but a low Recall (0.623): it rarely predicts "high interest", but is usually right when it does. Gaussian Naive Bayes with the 38 columns is the weakest (0.628): its independence assumption is strongly violated, because the six day-of-week columns are parts of the same variable and the two rolling averages are almost identical (correlation 0.983), so the same evidence is counted several times. With the four history columns Naive Bayes improves to 0.695 but, these columns being highly correlated, it does not beat the rule.

| Approach | Val F1 | Test Precision | Test Recall | Test F1 |
|---|---|---|---|---|
| 4. Decision Tree (`max_depth`= 7) | 0.741 | 0.782 | 0.622 | 0.693 |
| R. Random Forest regressor + threshold | 0.734 | 0.792 | 0.615 | 0.693 |
| 3. KNN ($k = 50$) | 0.715 | 0.788 | 0.602 | 0.683 |
| 5. Random Forest (`max_depth`= 8) | 0.715 | 0.796 | 0.619 | 0.697 |
| 1. Logistic Regression | 0.711 | 0.816 | 0.579 | 0.678 |
| 0. Rule: 14-day average $\geq$ threshold | 0.695 | 0.760 | 0.618 | 0.682 |
| 2b. Gaussian Naive Bayes (4 history columns) | 0.695 | 0.784 | 0.573 | 0.662 |
| 2a. Gaussian Naive Bayes (38 columns) | 0.628 | 0.715 | 0.555 | 0.625 |

**Table 4.15 Test results of the classification formulation after refitting on train+validation (sorted by validation F1).**

On the test set (Table 4.15), the selected Decision Tree reaches $F1 = 0.693$, only about 0.01 above the rule (0.682); the Random Forest regressor with the threshold obtains the same F1 (0.693). Logistic Regression (0.678) and both Naive Bayes models (0.662 and 0.625) fall below the rule. The Random Forest classifier is slightly higher on test (0.697), but the selection was made on validation and is not changed after seeing the test results. The confusion matrix of the selected Decision Tree on the test set contains 466 true positives, 130 false positives, 283 false negatives and 1,725 true negatives, giving Precision $= 466/596 = 0.782$ and Recall $= 466/749 = 0.622$. Overall, the classification formulation brings no improvement over the regression formulation, which provides both a number of views and an equally good yes/no answer.

## 4.14 Discussion

*Comparison with the common approach.*

The common `price2` task produces impressive but meaningless scores (100% and 92.91%), which disappear on unseen products (44.1%). In contrast, the forecasting task studied here has a clear business use, is evaluated on a future period that the models never saw, and is compared with the strongest simple rule. Its improvement is modest (4.8%) but measured honestly, and the leakage stress test shows that a careless implementation could have reported an inflated $R^2$ of 0.786 instead of 0.711.

*What drives accuracy.*

The largest gains come from the representation of the history: moving from yesterday's value to a 14-day average reduces the validation MAE from 2.666 to 2.125, whereas the best model reduces it only further to 1.963. The KNN experiment shows that poorly chosen inputs can make a reasonable algorithm worse than a simple rule. Ridge brings no gain in accuracy because the linear model does not overfit, although it illustrates how correlated features share weight.

*Choice of model.*

The Random Forest is selected because it has the lowest validation MAE and keeps the best test MAE. The gap with Linear Regression is small (2.023 versus 2.078 on test); if interpretability or speed mattered more than a 2.7% difference in MAE, Linear Regression would be a reasonable alternative.

*Limitations.*

Views are a proxy for demand, not sales. The data cover only about four and a half months of 2008 and the test period lasts 12 days, so the improvement is not sta-

tistically conclusive. Information on promotions, holidays and campaigns is missing, which explains the missed spikes and the slight under-forecasting in August. Because display position is fixed for each product, the model cannot answer causal questions such as the effect of moving a product to page 1. The one-day horizon suits daily merchandising but not stock planning. Finally, each model has only one tuned hyperparameter, selected on a single 16-day validation window, and the classification view reuses these hyperparameters and the default probability threshold of 0.5.

## 4.15 Chapter summary

This chapter has shown that the common `price2` task reflects leakage and memorisation rather than predictive ability. For the forecasting task, the 14-day average is a strong baseline; Linear Regression, Ridge, KNN (with relevant features only), Decision Tree and Random Forest all improve on it on validation, and the Random Forest is selected. On the test period it reaches an MAE of 2.023 and $R^2$ of 0.711, 4.8% better than the 14-day average, but it wins on only 7 of 12 days, which a fair coin would match 39.1% of the time; the improvement is promising but not yet conclusive. The error analysis reveals a slight under-forecasting in August and missed traffic spikes, the feature importances confirm the dominant role of the 14-day history and the weekly cycle, and the leakage stress test quantifies the danger of forgetting the one-day shift. Finally, the classification view confirms that regression is the most appropriate formulation of the problem.

# CONCLUSION

## General conclusions

This project applied the machine learning methods studied in the course to a concrete business question: forecasting, every evening, how many times each product of an online clothing store will be viewed on the next day. Working on the public *e-shop clothing 2008* clickstream dataset, we first showed that the task most often attempted on this dataset – classifying `price2` – does not measure real predictive ability: its 100% and 92.91% accuracies come from target leakage and from memorising 217 products, and accuracy on unseen products (44.1%) is below that of always predicting the majority class (51.2%).

We then reformulated the problem as one-step-ahead regression of daily product views. After removing a truncated last day and correcting an inconsistent category label, we built a complete product–day panel with true zeros, designed lag, rolling-mean and day-of-week features that use only past information within each product, verified them by a manual calculation, and evaluated four baselines and five regression models with a time-ordered train/validation/test split. The 14-day average turned out to be a strong baseline. The Random Forest, selected on validation, achieved a test MAE of 2.023 views per product per day and $R^2 = 0.711$, 4.8% better than the 14-day average. Because it won on only 7 of the 12 test days—a result that a fair coin would match about 39% of the time—we conclude that the model shows potential to improve the forecasts but that the evidence is not yet sufficient to claim with confidence that it is better than the baseline.

The experiments also showed that most of the accuracy comes from representing the history well rather than from model complexity, that KNN becomes worse than a simple rule when irrelevant features are included, that regularisation brings no gain in accuracy here, and that forgetting a single one-day shift would inflate $R^2$ from 0.711 to 0.786. An alternative classification formulation, which allowed Logistic Regression and Naive Bayes to be applied appropriately, gave no improvement over the regression formulation, confirming that regression is the most suitable way to pose the problem.

## Limitations and future work

The study has the following limitations:

- Views are only a proxy for demand; the data contain no purchases.
- The data cover about four and a half months of 2008, and the test period lasts only

12 days, so the improvement is not statistically conclusive.

- There is no information on promotions, holidays or campaigns, so sudden traffic spikes cannot be anticipated, and the model slightly under-forecasts in August.
- Display position is fixed for each product, so the model cannot answer causal questions such as the effect of moving a product to page 1.
- Each model has a single tuned hyperparameter selected on one 16-day validation window; the classification view reuses these hyperparameters and the default probability threshold.

Based on these limitations, we propose the following directions:

- Evaluate the models on several consecutive test windows (rolling-origin or walk-forward evaluation) and use the same procedure to select hyperparameters, in order to obtain more reliable conclusions [6].
- Add shop-level features (total views of the shop on the previous day, share of each product), dispersion and trend features (rolling standard deviation, difference between the 7-day and 14-day averages) and information from the clickstream itself (sessions, click order, country).
- Use loss functions designed for count data, such as Poisson regression [26], and gradient-boosting methods [27, 28].
- Forecast total views over the next seven days if the forecasts are to support stock decisions, and model the life cycle of products explicitly.
- Tune the probability threshold of the classification view according to the business objective (favouring Precision or Recall), and, if purchase data become available, forecast sales instead of views.

# REFERENCES


[1] M. Lapczynski and S. Bialowas, "Discovering patterns of users' behaviour in an e-shop – comparison of consumer buying behaviours in poland and other european countries," *Studia Ekonomiczne*, no. 151, pp. 144–153, 2013, special issue "La société de l'information : perspective européenne et globale".

[2] D. Dua and C. Graff, "UCI machine learning repository," University of California, Irvine, School of Information and Computer Sciences, 2019, http://archive.ics.uci.edu/ml.

[3] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1283–1318, 2022.

[4] K. J. Ferreira, B. H. A. Lee, and D. Simchi-Levi, "Analytics for an online retailer: Demand forecasting and price optimization," *Manufacturing & Service Operations Management*, vol. 18, no. 1, pp. 69–88, 2016.

[5] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[6] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[7] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[8] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.

[9] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021, https://otexts.com/fpp3/.

[10] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.

[11] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[14] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958.

[18] P. Domingos and M. Pazzani, "On the optimality of the simple Bayesian classifier under zero-one loss," *Machine Learning*, vol. 29, no. 2–3, pp. 103–130, 1997.

[19] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," *Journal of Machine Learning Technologies*, vol. 2, no. 1, pp. 37–63, 2011.

[20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[21] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.

[22] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, p. 25, 2007.

[23] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[24] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[25] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[26] A. C. Cameron and P. K. Trivedi, *Regression Analysis of Count Data*, 2nd ed. Cambridge University Press, 2013.

[27] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[28] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

# APPENDIX. REPORT–NOTEBOOK CORRESPONDENCE

The notebook `DS66B_group_3.ipynb` labels its models with identifiers and short Vietnamese descriptions. Table 5.1 gives the correspondence with the English names used in this report, so that every row of the result tables can be found in the notebook output. The figures of the report are drawn from the same variables as the notebook figures, with titles removed and axis labels translated into English.

| **Label in the notebook** | **Name in this report** |
|---|---|
| 0a. Baseline: lượt xem hôm qua (lag_1) | 0a. Baseline: yesterday (`lag_1`) |
| 0b. Baseline: cùng thứ tuần trước (lag_7) | 0b. Baseline: same day last week (`lag_7`) |
| 0c. Baseline: TB 7 ngày trước | 0c. Baseline: 7-day average |
| 0d. Baseline: TB 14 ngày trước | 0d. Baseline: 14-day average |
| 1a. Linear Regression (chỉ lag_1) | 1a. Linear Regression (`lag_1` only) |
| 1b. Linear Regression (tất cả đặc trưng) | 1b. Linear Regression (all features) |
| 2. Ridge Regression (alpha=0.1) | 2. Ridge Regression ($\alpha = 0.1$) |
| 3. KNN (k=50, 4 cột lịch sử + thứ) | 3. KNN ($k = 50$, history + day of week) |
| 4. Decision Tree (max_depth=7) | 4. Decision Tree (`max_depth`$= 7$) |
| 5. Random Forest (100 cây, max_depth=8) | 5. Random Forest (100 trees, `max_depth`$= 8$) |
| 0. Luật: TB 14 ngày $\geq$ ngưỡng | 0. Rule: 14-day average $\geq$ threshold |
| R. Random Forest hồi quy + ngưỡng | R. Random Forest regressor + threshold |
| 2a. Naive Bayes (38 cột) / 2b. Naive Bayes (4 cột lịch sử) | 2a. Gaussian Naive Bayes (38 columns) / 2b. (4 history columns) |
| Tất cả 38 cột / 4 cột lịch sử / 4 cột lịch sử + thứ | All 38 columns / 4 history columns / 4 history columns + day of week |

**Table 5.1 Model and feature-set labels in the notebook and in the report.**